SandEE: AN EDUCATIONAL MULTI-PLATFORM 2D OPEN WORLD SURVIVAL CRAFTING GAME FOR LUA AND PYTHON FOR ICT AND STEM STUDENTS OF YVERDON DE PESTALOZZI SCHOOL INC.

A Thesis
Presented to the Faculty of the
Information and Communications Technology Program
STI College San Jose Del Monte

In Partial Fulfilment
of the Requirements for the Degree
Bachelor of Science in Computer Science

Glenn Marco B. Accad
Adrian V. Gallano
Jefferson G. Mamauag
Jose Mari Christian V. Tayag

November 2026

ENDORSEMENT FORM FOR ORAL DEFENSE

TITLE OF RESEARCH: SandEE: An Educational Multi-Platform 2D Open World Survival Crafting Game for Lua and Python for ICT and STEM Students of Yverdon de Pestalozzi School Inc.

NAME OF DEVELOPERS: Glenn Marco B. Accad
Adrian V. Gallano
Jefferson G. Mamauag
Jose Mari Christian V. Tayag

In Partial Fulfilment of the Requirements
for the degree Bachelor of Science in Computer Science
has been examined and is recommended for Oral Defense.

ENDORSED BY:

Norvin Shaq F. Lintag
Thesis Adviser

APPROVED FOR ORAL DEFENSE:

Enoch James C. Tayanguna
Thesis Coordinator

NOTED BY:

Sheila Marie B. Llesol
Program Head

November 2026

APPROVAL SHEET

This thesis titled **SandEE: An Educational Multi-Platform 2D Open World Survival Crafting Game for Lua and Python for ICT and STEM Students of Yverdon de Pestalozzi School Inc.,** prepared and submitted **by Glenn Marco B. Accad; Adrian V. Gallano; Jefferson G. Mamauag; and Jose Mari Christian V. Tayag,** in partial fulfillment of the requirements for the degree of Bachelor of Science **in Computer Science,** has been examined and is recommended for acceptance and approval.

Norvin Shaq F. Lintag
Thesis Adviser

Accepted and approved by the Thesis Review Panel
in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science

Panel Member **Panel Member**

Lead Panelist

APPROVED:

Enoch James C. Tayanguna
Thesis Coordinator

Sheila Marie B. Llesol
Program Head

November 2026

ACKNOWLEDGEMENTS

The researchers would like to express their sincere gratitude to the individuals and institutions who made this study possible

To our thesis coordinator, Mr. Enoch James C. Tayanguna, for his support in the coordination, review, and defense process of the study;

To our thesis adviser, Mr. Norvin Shaq F. Lintag, for his guidance, patience, and constructive feedback throughout the planning, development, and writing of this study;

To the members of the Thesis Review Panel, for their valuable comments and suggestions that helped improve the quality of this work;

To our parents, guardians, and families, for their unwavering love, patience, and moral support; and

To our friends, classmates, and everyone who contributed in one way or another to the completion of this study, the researchers offer their heartfelt thanks.

Abstract

Title of research:	**SandEE: An Educational Multi-Platform 2D Open World Survival Crafting Game for Lua and Python for ICT and STEM Students of Yverdon de Pestalozzi School Inc.**
Researchers:	**Glenn Marco B. Accad** **Adrian V. Gallano** **Jefferson G. Mamauag** **Jose Mari Christian V. Tayag**
Degree:	**Bachelor of Science in Computer Science**
Date of Completion:	**November 2026**
Keywords:	**game-based learning, Lua, Python, educational game, programming education, survival crafting, open-world game.**

This study developed SandEE, an educational multi-platform 2D open-world survival crafting game designed to supplement the learning of Lua and Python among Grade 11 and 12 ICT and STEM students of Yverdon de Pestalozzi School Inc. The study was motivated by a needs assessment survey of 104 students, which showed limited familiarity with Lua but strong interest in learning it through interactive, game-based methods. Using the Agile methodology, the researchers designed and developed a single-player game that combines procedural world generation, survival and crafting mechanics, mission-based progression, and embedded Lua and Python interpreters. Player-written programs control Automatica and Algorithmica stations, allowing code to affect the game world in real time; syntax and runtime errors are reported in the in-game editor so debugging becomes part of the learning process. The game also includes themed Lua and Python dungeons, knowledge and quiz altars, a programming encyclopedia, and optional online accounts, cloud saves, and leaderboards. The system was evaluated through testing with IT students and expert review using selected ISO 25010 software quality characteristics. The IT student evaluation produced an overall weighted mean of 3.61, interpreted as Very High, while the IT expert evaluation produced 3.29, also interpreted as Very High. The results indicated that the game met its functional requirements and was rated favorably in terms of functional suitability, performance efficiency, and usability. The study concludes that SandEE can serve as a supplementary learning tool that makes programming practice more interactive, contextual, and engaging for beginner learners. Recommendations for further development include lowering the default audio level, adding full-screen and interface-scaling options, improving the tutorial and object-interaction instructions, reducing mission text overload, strengthening security and data integrity, and adding more story or lore elements.

TABLE OF CONTENTS

Acknowledgements **iv**
Abstract **v**
List of Tables **vi**
List of Figures **9**
List of Appendices **10**
Table of Contents **vi**
Introduction **11**
Background of the problem 13
General Problem 14
Specific Problem 14
Overview of the current state of the technology 16
Objectives of the study 19
General Objectives 19
Specific Objectives 19
Scope and limitations of the study 21
Review of related literature/studies/systems 35
Foreign Literature 35
Local Literature 36
Foreign Studies 37
Local Studies 39
Foreign Systems 40
Local Systems 41
Synthesis 42
METHODOLOGY **44**
Conceptual Framework 47
System Design Specification 48
Diagrams 55
Hardware/Software 62
Calendar of Activities 64
Budgetary Estimate 66

Results and Discussions **69**
Overview of the results 69
System Implementation 70
Data Gathering Procedure 71
Statistical Treatment of Data 72
User Feedback 73
Summary of IT Student Evaluation Results 79
Expert Feedback 80
Conclusion **84**
Summary 84
Conclusions 85
Recommendations 87
References **88**
APPENDIX A. GANTT CHART 91
APPENDIX B. ACTUAL THESIS EXPENSES 94
APPENDIX C. USER'S MANUAL 96
APPENDIX D. SURVEY 157
APPENDIX E. SOURCE CODE 167
APPENDIX F. CURRICULUM VITAE OF RESEARCHERS 188

LIST OF TABLES

1 Functional Modules and Descriptions 51
2 Hardware 64
3 Software 65
4 Budgetary Estimate 68
5 Mean Range Interpretation Legend 72
6 Functional Completeness 73
7 Functional Correctness 74
8 Functional Appropriateness 74
9 Time Behaviour 75
10 Resource Utilization 75
11 Capacity 76
12 Appropriateness Recognizability 76
13 Learnability 77
14 Operability 77
15 User Error Protection 78
16 User Interface Aesthetics 78
17 Accessibility 78
18 Summary of IT Student Evaluation Results 79
19 Functional Suitability - IT Expert Evaluation 80
20 Performance Efficiency - IT Expert Evaluation 80
21 Compatibility - IT Expert Evaluation 81
22 Usability - IT Expert Evaluation 81
23 Reliability - IT Expert Evaluation 81
24 Security - IT Expert Evaluation 82
25 Maintainability - IT Expert Evaluation 82
26 Portability - IT Expert Evaluation 82
27 Summary of IT Expert Evaluation Results 83
28 Actual Thesis Expenses 95

LIST OF FIGURES

1 Agile Methodology 51
2 Conceptual Framework 64
3 System Architecture 65
4 Data Flow Diagram Level 0 68
5 Data Flow Diagram Level 1 72
6 Entity Relationship Diagram 73
7 Hierarchical Chart 74
8 Account Authentication Flowchart 74
9 Gameplay Flowchart 75
10 Dungeon Gameplay Flowchart 75
11 SandEE Gameplay Screenshot 76

LIST OF APPENDICES

A Gantt Chart 88
B Actual Thesis Expenses 91
C User's Manual 93
D Survey 146
E Source Code 156
F Curriculum Vitae of Researchers 176

INTRODUCTION

In recent times, especially in information communication technology, programming has been the focus of computing education. The recent advent of several tools has made coding more accessible than ever (Becker, 2023). Programming has been a challenge for learners. The proliferation of digital technologies has made programming skills increasingly important across various disciplines. This shift extends beyond traditional computer science domains (Lin, et.al., 2025). According to industry reports, the demand for Lua developers has been steadily increasing in recent years. As more companies turn to Lua for its speed, simplicity, and versatility, the need for skilled Lua developers has never been higher. In particular, the gaming industry has seen a rise in Lua-related job openings, with many companies looking for developers proficient in Lua scripting (Crudu, 2024). The Lua scripting language is a fast, procedural programming language it was designed to be highly customizable with simple API in the C programming language so that programmers could easily make changes to fit their needs. Python has a simple syntax that allows developers to write programs with fewer lines compared to other programming languages Additionally, Python as a scripting language has been steadily gaining traction since its inception. Its simplified syntax allows learners to easily learn and grasp the coding structure of the scripting language (Python, 2021).

Yverdon De Pestalozzi School Inc. is a private senior high school offering STEM and ICT strands, dedicated to developing students' technical skills, academic competence, and readiness for future careers through a supportive and well-structured learning environment.

According to Lampropoulos (2023), Game-based learning emerged as a useful education approach that can be facilitated and enrich the learning process especially in students. This approach creates a motivational virtual learning environment that consciously engages and encourages students to actively participate in educational activities. This approach has also been used extensively in the field of programming, allowing an alternative approach to learning various programming concepts.

In a survey conducted to ICT and STEM Senior High School students of Yverdon De Pestalozzi School Inc., a total of 104 respondents participated, consisting of 22 students from Grade 11 ICT (21.2%), 32 from Grade 11 STEM (30.8%), 16 from Grade 12 ICT (15.4%), and 34 from Grade 12 STEM (32.7%). The results revealed that most students demonstrated average familiarity with the Lua programming language, suggesting that it is relatively new to many respondents. In addition to this, the majority expressed positive attitudes toward learning Lua through supplementary game-based activities, with 81 (78.9%) agreeing that such an approach would improve their understanding of programming concepts and 76 (73%) showing motivation to explore Lua using an interactive approach. On the other hand, Python received generally favorable feedback, with 84 (80.8%) of participants finding its syntax and structure easy to understand and applicable to programming tasks. Overall, the findings indicate that while students are less experienced with Lua compared to Python, they show strong enthusiasm and willingness to learn it through interactive and game-oriented methods that enhance engagement and learning outcomes. As a result, introducing a gamified approach of learning Lua and Python is applicable to the students at the school.

SandEE is an educational 2D open world survival crafting game for Lua and Python. It introduces several programming concepts related to Lua and Python. Furthermore, SandEE also includes gameplay that can serve as an exercise to advanced users of Lua and Python. The proposed study offers a supplementary learning approach for Lua and Python by slowly introducing its syntax and structure. The game embeds interpreters for Lua and Python in which code that is run impacts the game environment to show visual feedback, as well as to provide a realistic basis to the player as the interpreters will give information accurate to its real use-case. Progression is achieved through the mission system, which guides the player to progress in learning. These missions will eventually lead to rewards to further gratify the player and give them feedback on their progress.

Background of the problem

The rapid advancement of technology has significantly transformed educational methodologies. Traditional learning, which often emphasizes theoretical concepts and static exercises, struggles to fully engage students or prepare them for real-world problem-solving. In contrast, educational games have emerged as an innovative approach to learning, combining interactivity and engagement with skill development. Popular examples like Duolingo demonstrate how gamified systems can make learning both effective and enjoyable. Game-based learning in programming has shown the potential to enhance logical reasoning, problem-solving, and creativity through active participation.

To better understand the learning preferences of ICT and STEMS students, a survey was conducted among the grade 11 and 12 ICT and STEM students of Yverdon De Pestalozzi School Inc. The results revealed that most respondents had average familiarity with the Lua programming language, with the majority rating their understanding between 2 and 4 on a 4-point scale. However, despite this lack of experience, 82 (78.9%) of respondents rated 2, 3, and 4 when asked how helpful and motivating it would be to learn Lua through game programming. For comparison, students showed higher familiarity with Python, averaging between 2, 3 and 4 in both ease of use and coding level. Yet even those comfortable with Python expressed strong interest in learning programming concepts through interactive, game-based methods. These findings indicate that students are more receptive and motivated when learning involves playing, which shows what happens inside the game, experimentation, and engagement rather than passive listening and reading books.

To address these findings, the proposed game introduces an immersive, game-based platform that integrates survival and crafting mechanics within an open-world sandbox-type environment. Through hands-on coding challenges embedded in gameplay, it aims to foster critical thinking, logical reasoning, and practical programming skills. This approach seeks to make programming educational, creative, experiential, contextual, and enjoyable.

General Problem

How to develop a game to supplement the high interest in learning Lua and Python through a game?

Despite low familiarity with ICT and STEM students, survey results have shown that interest in learning Lua and Python is high. Furthermore, the results have also shown an increasing interest in learning these languages using a more interactive and engaging approach through a game.

Specific Problem

- **How to develop a game that enhances programming skills by providing hands-on learning where concepts are learned through experience?**

  Many ICT and STEM students have shown a strong interest in learning programming languages such as Lua and Python, particularly through interactive and engaging methods. Based on the survey conducted, there is a clear inclination toward learning through game-based experiences that allow the practical use of programming concepts. Therefore, this study focuses on developing a game that enhances programming skills by providing hands-on learning where concepts are learned through experience, serving as an innovative approach to support growing interest in programming.

- **How to develop a game that provides direct visual feedback as players learn and apply programming concepts?**

  Students often show greater engagement and understanding when they can immediately see the results of their actions while learning. In programming education, providing direct visual feedback allows learners to observe how their inputs affect outcomes in real time, creating a more interactive and immersive experience. Therefore, this study aims to develop a game that provides direct visual feedback as players learn and apply programming concepts, promoting better comprehension and active participation through visual interaction.

- **How to develop a game that realistically responds to player errors by embedding Lua and Python interpreters within the system?**

  To create a more authentic and educational programming experience, it is important for a game to respond realistically to player inputs, including coding errors. Embedding Lua and Python interpreters within the system enables real-time evaluation of player code, allowing the game to simulate how actual programming environments handle mistakes.

- **How to develop a game that introduces the programming structure and syntax of Lua and Python?**

  Learning programming languages such as Lua and Python requires a clear understanding of their syntax and structural rules. Introducing these foundational concepts through an interactive approach can make learning more engaging and effective.

Overview of the current state of the technology

In recent years, advancements in teaching coding through educational and engaging means have been accomplished. There are ways, including game-based learning and digital courses, that provide plenty of valuable knowledge on how to code to solve problems and perform tasks.

Philippine Coding Camp (2023)

Such examples include the Philippine Coding Camp. These are digital learning bootcamps that primarily focus on teaching students of any age by making the courses simpler, welcoming, and open to beginners and experienced learners alike. Both provide structured courses that teach the fundamentals and concepts of programming, which they can later apply to what they have learned through exercises and challenges. They use platforms such as Roblox as a supplement and make learning more engaging. As learners progress through the courses, new concepts are slowly and simply introduced for them to better understand and retain what they are learning.

While these courses are structured, there are limitations that can be addressed. Such as the focus is only centered on the topics of the course. By making the process more hands-on, it removes the restriction of complying with the structure of the course and letting the learners learn in their own way with similar guidance.

Logiscool (2025)

Logiscool is a similar coding camp established worldwide and has a digital platform where they also teach digital courses. They provide programs where in each, they teach digital literacy, AI, and coding. They integrate gaming platforms such as Roblox and Minecraft into their learning curriculums to make learning more interesting. In addition, they also use artificial intelligence to support teaching. Part of the programs they teach are emerging languages such as Python and Lua. Python is taught within topics such machine learning, and Lua is primarily used in Roblox to create games and experiences. Logiscool's main target is younger users but are still open to everyone.

Logiscool's programs thoroughly teach its learners about the topics, provide and hones skills such as critical thinking and creativity. But by creating a more engaging way to teach, similarly to how they use games, learners can better learn and apply their skills into practical applications.

Hello World (2024)

Hello World is a game that primarily teaches Java and C#. It teaches fundamental concepts of these two-programming languages and integrates them into gameplay. Throughout Hello World, introductory lessons are provided, then the player must use what they have learned to progress. The game then adds the challenge of using different concepts to solve a given problem. It teaches through direct engagement with languages since it is incorporated into the gameplay. The strength of the system comes from the interactive process of learning languages.

This study provides aims to recreate the process of learning through interaction with languages, but uses two emerging languages, Lua and Python. Java and C# are already commonly taught in the field, but there is a growing need in the industry that seeks developers with knowledge of Lua and Python.

Code Combat (2025)

CodeCombat is a digital gaming platform that primarily focuses on teaching fundamentals of programming and applying it into game scenarios. Ozaria is built from CodeCombat, where Python is one of the main focuses. They have a custom built-in engine and interpreter that helps beginners in learning how to code. This platform can be integrated in school settings, allowing students to share their projects with others and teachers to grade their work. CodeCombat works by teaching students first, then placing them in game scenarios where their knowledge is put to the test. Ozaria expands this by creating a storyline that progresses with the learner's skill and knowledge on the subject.

There are limitations to this, however. While this type of learning is more engaging, it lacks the structured format that courses provide. They teach first then send the learners into performing tasks. The study aims to bridge the gap between digital courses and gamified learning by blending the two ways of learning into a consolidated process. By merging the structured format of courses and the hands-on engagement that games provide, this creates a foundation for students to learn programming concepts in a unified form.

CodinGame (2025)

CodinGame is an online gaming platform that uses coding as its main method of playing. It challenges developers to use their skills in competing against others across different types of games. It encourages the application of what the user knows about programming. It features several programming languages such as Lua and Python, and developers must utilize their skills of the language throughout CodinGame's challenges and competitions. This type of learning helps developers see what they can still learn and improve to solve future challenges and tasks they may face.

CodinGame is limited since it primarily focuses on competition against other players. It doesn't provide an exact way to teach; it only points out what is lacking when it comes to knowledge. This type of learning is limited, hence creating a game that teaches, and rewards players gives a sense of accomplishment and progress that provides a better experience compared to the competitive nature of CodinGame.

Objectives of the study

General Objectives

To develop a game to supplement the high interest in learning Lua and Python through a game.

The proponents of the projects aim to develop a game with the goal of supplementing the high interest of learning the Lua and Python programming language through a game approach by integrating engaging gameplay with learning activities.

Specific Objectives

- **To develop a game that enhances programming skills by providing hands-on learning where concepts are learned through experience.**

  The proponents of the project aim to create an interactive learning environment where students can apply programming concepts directly within gameplay. Specifically, the game will encourage learners to solve coding challenges that simulate real-world scenarios, promote active problem-solving, and experiential learning rather than passive observation.

- **To develop a game that provides direct visual feedback as players learn and apply programming concepts.**

  The proponents of the project plan to use the game's environment to display visual feedback based on the user's coding input. Specifically, the player's actions and written code will directly affect the environment, allowing them to see how Lua and Python interpret their logic in real time. This feedback helps learners understand programming flow and identify errors through immediate, intuitive results.

- **To develop a game that realistically responds to player errors by embedding Lua and Python interpreters within the system.**

  The proponents of the project aim to ensure that the game reacts authentically

to user errors by integrating Lua and Python interpreters. Specifically, this enables the application to detect and display syntax or runtime errors accurately, providing a realistic debugging experience. This approach helps students learn from their mistakes and develop stronger problem-solving and error-handling skills.

- **To develop a game that introduces the programming structure and syntax of Lua and Python.**

  The proponents of the project intend to integrate Lua and Python learning modules within the game. Specifically, the application will include coding activities that demonstrate syntax rules, control structures, and logical operations in both languages. This allows users to understand and practice programming concepts naturally while progressing through the game.

Scope and limitations of the study

- **User Account**
    - Each user can create a unique account using their own username and password.
- **Cloud Saves**
    - The save file of each user will be uploaded to the internet every time the game saves.
    - The save file of each user is accessible multi-platform.
    - Cloud saves prioritize the save with the highest amount of stars to prevent conflict.
- **Offline Mode**
    - Allow the player to play the game offline.
    - Players can switch to online mode from offline mode.
- **Online Mode**
    - Allows the player to access the leaderboard and cloud saves.

- **Main Menu**
    - Continue
        - This allows the player to continue from their recently saved world.
    - Worlds
        - This will redirect the user to the saves menu to select their desired save or start a new save.
        - If a player instanced a world on a device, the game locks that specific world. The releasing device uploads the latest save and releases the lock. Other devices receives the new revision of the world and unlock status on their next sync or when worlds screen is refreshed.
    - Leaderboards (Online Mode Only)
        - List the rankings of player's acquired blue and yellow stars.
    - Cloud Save Management (Online Mode Only)
        - Allow the player to manage cloud saves.
    - Account (Online Mode Only)
        - Allow overview of the account and allow logging out.
    - Settings
        - This will redirect the user to the settings panel to adjust the game's sound volume.
    - Quit
        - This will close the game.
- **Settings Panel**
    - Volume Sliders
        - BGM Volume

- Adjusts the volume for the background music of the game.

        - Effects Volume
            - Adjust the volume for the sound effects of the game.
    - Fullscreen
        - Sets the game to full screen window.
    - Credits
        - Lists the developers and individuals who are part of the game development along with references to the materials and software used.

- **Worlds Menu**
    - World Creation
        - Lets the player name the world before creation.
    - Delete
        - Deletes the chosen save file upon confirmation.
    - Rename
        - Allows the player to rename their worlds.
    - Save Slot
        - Loads the chosen save file to continue its progress.

- **Game Mechanics**
    - Procedural Map Generation
        - Single player maps are automatically created using algorithms instead of being manually designed which allows each environment to be unique.
    - Waypoints
        - Allow teleporting to certain areas.

- Missions
    - A list of gameplay objectives that gradually guides the player throughout the entire game.
    - Dungeon missions are separated into distinct levels of difficulties which encompasses a gradual flow of progression into learning programming.
        - Easy mission introduces basic concepts.
        - Medium missions combine concepts.
        - Hard missions teaches advanced concepts.
    - Completing missions will give the player stars and the stars they have will determine their rank.
- Ranks
    - Ranking system will allow players to display their programming capabilities in the game.
    - Ranks can be increased by completing dungeons and earning blue stars.
- Completion Badge
    - Granted to the player upon completing extra dungeon missions. Only given once.
- Leaderboards
    - Sort available players by their rank.
    - Rankings are based on the blue stars the player has.
- Crafting
    - The user will be crafting various items that improve their current state or progress the missions provided.
    - The user can craft multiple categories of items which include

materials, interactive objects, weapons, and foods. Some of which can have additional functionality using programming.

        - The users can craft items to build and progress.

- Fishing
    - Players can use fishing rods to catch fishes in bodies of water.
- Farming
    - Players can use tools to collect seeds and build farmland near water.
- Items
    - Materials
        - It can be used to craft materials and other craftable items.
    - Interactive Objects
        - Crafting Stations
            - Where the user can craft items.
        - Furnace
            - Allows the player to smelt ores and cook food using either wood or charcoal as fuel.

- Chest
    - Allows the player to store items in a limited capacity.
- Color Crafter
    - Allows the player to craft items with custom RGB colors.
    - Displays the hex number for the selected block.
    - Player can select between hex or RGB.

- Weapons
    - Swords can be used to deal damage to enemies.
- Tools
    - Tools such as the pickaxe, axe, and fishing rod can be used for specific purposes.
- Food
    - Can be used to heal health.

- Dungeons
    - Randomly generated areas that serve to give major progress for the user, challenging their skills in Lua or Python programming to unlock more mechanics.
    - Rewards the player with blue stars upon completion.
    - Serves as a guide to player's progress.
    - Dungeons allows hints to help players when they get stuck.

        - Randomly spawns successively throughout the world and serve as the dungeon mechanic of the game. The next dungeon spawns after the last one are completed.
        - Includes quizzes that can be solved openly depending on the user.
    - Caves
        - Large Caves
            - Generates in the overworld. Provides finite source of stone and ores.
        - Pixie Caves
            - Special caves with resources that regenerate after a period of time.
        - Friendly Mobs
            - Passive mobs like sheeps and wolves wander the overworld.
    - Enemies
        - Spawns randomly on the overworld and in dungeons.
        - Enemies that spawn on dungeons do not adhere to the day and night cycle.
        - Headless Fish
            - Spawns in the overworld during the night and attacks by homing and dashing into the player.

- Desert Worm
    - Spawns in the overworld regardless of daylight. Spawns and hides in the desert and attack the player when approached.
- Hitman
    - Spawns in the dungeon. Fires projectiles and can charge straight into the player to deal damage.
- Pixie
    - Spawns in the Pixie Caves and attacks the player on sight.
- Brass Prowler
    - Python dungeon enemy.
- Cipher Warden
    - Python dungeon enemy.
- Rune Sentinel
    - Lua dungeon enemy.
- Vesper Acolyte
    - Lua dungeon enemy.
- Cinder Ooze
    - Overworld enemy.
- Gravebell Shambler
    - Overworld enemy.

- Day and Night Cycle
    - Determines enemy spawning rate and difficulty.
    - Enemies deal less damage during daytime and do normal damage at night.
- Live Interpreter
    - Serves as a built in IDE for the coding function in the game for both Algorithmica and Automatica.
    - Syntax Highlighting which will highlight the errors in programming syntax.
    - Interprets the code that is written by the player.
- Automatica
    - Robot-like craftables that allow the user to automate certain processes by programming it through Lua, through the Lua-focused dungeons.
    - Automaticas can be used to automate interaction with objects and aid the player in combat.
    - Automaticas required placed wires to interact with stations.
    - Automaticas have the option to be renamed.
- Algorithmica
    - Allows the user to automate actions in crafting stations, furnaces, and color crafters.

- LUA (Live Unit Application)
    - An advanced Automatica non-player character that introduces the user to the Lua language.
- Py-san
    - An advanced Algorithmica non-player character that introduces the user to the Python language.
- Autosave
    - The game will automatically save the player's progress periodically and during state changes such as entering caves and dungeons, as well as when the player quits the game.
    - The game also autosaves every 60 seconds.
- Build Mode
    - Can be toggled on or off to place down blocks.

- **Desktop and Mobile Application**
    - HUD (Heads-Up Display)
        - Pause
            - Allow the user to temporarily pause the game, showing the Pause Menu Panel.
        - Pause Menu Panel
            - Resume
                - Resumes the current game.
            - Settings
                - Shows the settings panel.
            - Automatica management.
                - Allows the player to manage the automaticas.
            - Quit
                - Saves the game and directs the user to the Start Page.
        - Mission Panel
            - Shows the user the current missions they can do to progress.
        - Health Bar
            - Shows the user their current health.
        - Stamina Bar
            - Shows the current stamina, which drains when running.
        - Hotbar
            - Show the user their current held items.
        - Inventory
            - Shows the user their extra space and a smaller crafting area.

- Programming Encyclopedia
    - Contain details about programming concepts for Lua and Python learned through dungeons and custom functions for Algorithmica and Automatica.
- Code Editor Panel
    - A panel that prompts the user to code in Lua or Python to either progress in a dungeon or customize their Automatica and Algorithmica.
- Crafting Panel
    - A panel for more advanced crafting, unlockable when the user crafts the crafting station.
- Map
    - A representation of the world that shows the overworld, cave, and dungeons.

- On-screen Joystick
    - Allows movement for mobile users.
- Interact Button
    - A general button that changes depending on the items near the player's character.
- Build Mode Button
    - Can be toggled on or off if the player wants to build.

Limitations

- **The game only runs on Windows 10 – 11 and Android 8.0 and above.**

  Ensures the game performs consistently across devices with modern hardware and software support. Older systems may lack the necessary performance, security updates, or engine compatibility required by Godot.

- **The game only supports Lua and Python for its languages.**

  This limitation is based on both practicality and accessibility. Lua and Python are in high demand across education, game development, and general programming, making them valuable languages for learners to practice. By restricting the system to these two languages, the game maintains a focused learning environment that is accessible to new programmers while still relevant to real-world applications.

- **The game only supports single player.**

  This limitation ensures that each student's learning experience remains individualized and self-paced. By focusing on single-player gameplay the game allows students to develop their skills independently and maintain consistent progress without distractions from multiplayer interactions.

- **The game only references the 2025 version of official documentation for Lua 5.1 and Python 3.11.**

  This limitation ensures that all learning materials are consistent, stable, and aligned with a single verified source. Lua 5.1 and Python 3.11 have well-established, mature, and consistent syntax, making them reliable for teaching without frequent changes or breaking updates. By locking the game's references to the 2025 documentation versions, students receive clear and uniform guidance.

Review of related literature/studies/systems

Foreign Literature

According to Crudu (2024), Lua developers are in high demand because it is currently widely used in the programming industry. Lua is efficient, flexible, scalable, and cost effective to apply in any programming field. According to the TIOBE Index, Lua is currently ranked as the 32nd most popular programming language. Furthermore, in a survey conducted by Stack Overflow in 2024 it is among the top 10 most loved programming languages.

According to Becker (2022), Generating high-quality learning resources is very time consuming and typically requires a high level of expertise therefore, games have been used as a learning tool for centuries; they have become more prominent in educational settings. When it comes to learning, being focused and immersed can massively improve a student's experience.

According to Viduka (2021), Python is preferred by students and professionals alike due to its ease of use, availability, efficiency, portability, and extensibility. It is a common choice suitable for education. Many schools starting from primary to secondary, all the way to universities prefers teaching Python as primary programming languages.

Local Literature

Computer science software development evolves rapidly; innovative learning platforms transform how instructors teach and how aspiring developers learn programming. Several other solutions integrate interactive web based on learning programming languages. However, these methods of studying are not applicable to everyone. This is where a game-based solution can complement the traditional way of learning (Horlador, et., 2024)

The rapid development of computer science education signifies that traditional methods of learning are being challenged. Conventional approaches often are lacking when it comes to engaging students or providing practical application of programming concepts (Horlador, et., 2024). Traditionally, a beginner developer sits inside a classroom, listening to an instructor as they explain related programming concepts. However, there are instances wherein the beginner developer cannot follow along with the instructor's sample codes (Horlador, et., 2024).

The current education system is outdated in the sense that it cannot keep up with the demands of a fast-paced and information-rich world outside of the classroom. In response to these challenges, innovative learning platforms can be applied to transform the way beginner developers usually learn (Horlador, et., 2024).

In today's rapidly evolving landscape, mastering the concepts of Information Technology is crucial for the students aspiring to enter the technology industry (Abat, et al., 2024). Software development is fundamental to the innovation and efficiency of the applications and solutions tailored to specific needs and demands (Abat, et al., 2024). This caused the current curriculum to include programming languages such as Lua and Python (Abat, et al., 2024).Despite the abundance of academic materials and tutorials regarding different types of programming languages, most of the students still face difficulties in learning the fundamentals of programming and how to apply them. By implementing interactive game-based learning, it allows students to learn using an alternative method of learning that is both fun and educational. (Abat et al., 2024).

Foreign Studies

According to Eltomi (2025), the use of Lua programming language allows the game developers to easily implement core gameplay mechanics such as character movements, physics, and collision detection. This study also identified the problems that game developers encountered such as no prior knowledge in Lua programming language, Game size and complexity, and bad game designs. In relation to the study, it explains the use of Lua programming language in game development. However, the proposed study lacks the idea of showing how the actual Lua language is used in game development.

In a study done by Verbovetsksvi (2025), The authors identified the basic requirements for a game application, such as relevance to educational objectives, incorporation of game mechanics, the ability to write and debug code in the application, individualized passing of the game, and the ability to report and rate learning outcomes of students. The paper contains some fragments of the game workspace and code of a gaming application.

Several studies compare the use of Python and Lua in game development, highlighting their distinct advantages. Python, with libraries like Pygame, is favored for its simplicity and extensive community support, making it ideal for rapid prototyping and educational game development (Oleksiuk et al., 2024). However, it tends to have slower performance compared to Lua. Lua is widely used in professional game engines due to its lightweight nature, fast execution, and seamless integration with C/C++ code, making it suitable for performance-critical game scripting (MoldStud, 2025).

Tyagi's study (2021) compares developing a 2D Flappy Bird game in both Python (Pygame) and Lua (LOVE engine), showing that while Python is easier for beginners, Lua provides better performance and embedding capabilities for commercial projects. Other research agrees that Lua excels in embedded scripting and resource-sensitive environments, whereas Python is often used for game prototyping, tooling, and educational purposes.

In conclusion, these studies demonstrate that the choice between Python and Lua depends on the project goals, with Lua being preferred for efficiency and embedding, and Python for ease of use and broad application.

Local Studies

According to a study done in the University of Mindanao-PALMA Cluster Campus Philippines aimed to determine the effectiveness of game-based learning, based on the findings it can be affirmed that implementing game-based learning strategies is highly effective, resulting in a notable enhancement of learners' academic performance compared to conventional teaching methods. Moreover, the results encourage teachers to revisit their instructional materials and integrate game-based learning materials aligned with specific learning objectives. Additionally, the study provides substantial evidence that game-based learning significantly enhances academic performance compared to traditional instructional resources (Cezar, 2024).

According to Cezar (2022), a study was done to measure the effect of digital game-based learning. Educators and authors argued that video games are constructed on well-developed theories to engage players. The gathered data were interpreted and analyzed; the results revealed that the experimental group achieved higher scores in their academic performance test. The findings showed a significant difference in the performance for students engaging in digital game-based learning.

According to Justo et al. (2022), a study was done to analyze students' perspectives of game-based learning. The results have been proven the game-based learning has a more significant influence on retention and class interaction when this technique is used. As a result, the findings say that game-based learning aids information acquisition and boosts student engagement through increased class interaction.

Foreign Systems

CodeCombat (2024) is a game-based learning platform that teaches languages such as Python through interactive, role-playing gameplay. Its custom-built coding engine and interpreter make it beginner friendly. The platform includes project-sharing features and teacher grading systems, making it usable in both personal and educational contexts. According to Ahlstrom (2025), the system not only simplifies coding, but they execute it in a method that emphasizes having fun. CodeCombat demonstrates how games can effectively serve as a medium for learning programming concepts.

OmegaCrafter (2025) is an open-world survival craft game set in an underdevelopment game world that is being attacked by a mysterious program. Grammi, the players' programmable sidekick can be programmed with various actions and tasks to help with mining, logging, moving materials, and crafting items (Preferred Networks, 2025)

The Farmer Was Replaced (2023) is a game for Python-like language to fully automate various farming tasks that would otherwise be time consuming. (Herzog, 2023). Unlike most programming games, it is not divided into distinct levels that players have to complete but instead features continuous progression. Programming is done in a simple language similar to Python. The game is designed to provide basic programming concepts to the player (Herzog, 2023).

Local Systems

Philippine Coding Camp (2023) is a digital learning center that teaches Filipino students how to code through structured, hands-on programs. The system provides learning courses which teach Lua as one of the foundations for beginner and younger students while also introducing Python later as a part of the game. In relation to the study, the system highlights accessibility in programming education while sustaining engagement and interactivity that traditional digital courses may not fully achieve.

Logiscool (2025) is a digital learning platform that focuses on coding and digital literacy education. Its courses integrate popular platforms to provide engaging environments to enhance coding concepts. The system demonstrates the effectiveness of combining fun and interactivity with education.

Hello World (2024) is a game that teaches the fundamentals of Java and C#. Players apply their skills into gameplay where challenges are introduced. The game progresses based on what the players have learned so far, from introduction to more advanced topics. It is a practical application of game-based learning in the growing industry of programming.

Synthesis

The reviewed foreign and local literature, studies, and systems collectively emphasize the growing importance of game-based learning in programming education. Foreign authors (Becker, 2022; Crudu, 2024; Viduka, 2021) highlighted how games foster immersion and focus, while also pointing to the industry relevance of programming languages such as Lua and Python. Local studies (Horlador et al., 2024; Abat et al., 2024) reinforced these ideas by showing how interactive, game-based methods can complement or even surpass traditional classroom instruction, especially in engaging students who struggle with conventional approaches. Moreover, both foreign and local systems (e.g., CodeCombat, 2024; Philippine Coding Camp, 2023) demonstrated practical applications of integrating coding into interactive platforms, providing evidence that game-based solutions are not only feasible but effective in real-world educational settings.

However, despite these advances, the reviewed literature also reveals a gap: while many systems teach Python or Lua through games, few explicitly focus on bridging the fundamentals of programming with practical coding immersion tailored for beginners in the Philippine context. This gap underscores the significance of the present study, which seeks to design a game-based solution that leverages the strengths of Lua and Python while addressing the challenges faced by novice developers in traditional learning environments. In the context of learning, both data gathered from foreign and local literature are that gaming tools have been used as a powerful supplementary tool for learning and can be applicable into learning complex topics such as concepts of programming (Becker, 2022; Horlador, et al., 2024).

The difference between the gathered information from foreign and local literatures are that in foreign literature, it has been found that games have been increasingly prominent in educational settings since when it comes to learning, students can be more focused and immersed (Becker, 2022). Local literature on the other hand has found that traditional methods are becoming outdated and are challenged by more innovative means of learning, especially considering the context of teaching complex topics in Computer Science (Horlador, et al., 2024).

Foreign studies done by authors (Eltomi, 2025; MoldStud, 2025; Tiyagi, 2021) gathered that Lua and Python are applicable in implementing features much easier due to each of the advantages that the two languages have. Lua is a lightweight language and integrates with C and C++ code while Python is more beginner-friendly and applicable for prototyping and tooling.

Local studies proven by authors (Cezar, 2022; Justo, et al., 2022) found that gamified learning is an effective method of learning as it has been gathered that the group that was engaged in gamified learning scored better in comparison to the group that did not. Based on what the study has gathered from the local and foreign studies, the goal is to implement gamified learning into teaching students the concepts of Lua and Python as these two languages have been shown to be appealing for beginners to learn (Oleksiuk, et al., 2024; MoldStud, 2025).

The proposed game aims to be unique by implementing the methods of gamified learning into teaching the concepts of Lua and Python in such a way that it acts as a supplement alongside traditional learning methods. It acts to teach students in a more engaging way. It does so by applying concepts and ideas that the foreign and local literature and studies have gathered and combine them in such a way that it introduces a specialized method of teaching students various programming concepts. Lua and Python were chosen as the two programming languages to teach because of their simplicity and utility, making them appealing to teach.

METHODOLOGY

Agile Methodology

Agile methodology is an iterative project management approach in which development is divided into short cycles called sprints. It relies on teamwork, early and continuous communication, and repeated review, allowing the game to evolve based on feedback rather than following a single sequential design process. For SandEE, the methodology was organized into goals, ideation, planning, sprint, iteration, and production.



Figure 1. Agile methodology

Goals

The goals phase defined the objective of developing an educational 2D open-world survival crafting game that teaches Lua and Python to Grade 11 and 12 ICT and STEM students. The target users were identified, and the core game functions were initially conceptualized, including world exploration, crafting, missions, dungeons, and in-game programming.

Ideation

The ideation phase focused on generating features and solutions for the game. The team explored programming challenges, survival mechanics, dungeon designs, and interface ideas, then selected the concepts that best supported the learning objectives and the available resources.

Planning

The planning phase converted the selected ideas into a structured plan. Tasks were prioritized, sprint timelines and deliverables were set, and responsibilities were assigned. This phase ensured that each sprint had a clear goal and that the team understood the scope of the work.

Sprint

The sprint phase was the core of the development process. Each sprint lasted a fixed period, usually one to four weeks, and produced a playable feature or module such as world generation, crafting, missions, or the code editor. Every sprint ended with testing and review of the delivered component.

Iteration

After each sprint, the team reviewed feedback, test results, and stakeholder input. Adjustments, bug fixes, and feature enhancements were made, and the revised work was carried into the next sprint. This iterative cycle supported continuous improvement and allowed the game to respond to user needs.

Production

Once the sprints and iterations were complete, the final game was prepared for

production. The team conducted final testing, documentation, and optimization, and then deployed the Windows and Android builds. Maintenance and future updates were scheduled to address issues and additional feature requests.

Conceptual Framework

The study is guided by the input-process-output (IPO) model, a widely used framework for describing how a system transforms identified needs into a working solution and measurable results. The model is appropriate for SandEE because the game is both a software product and a learning intervention: its design must respond to the learners' needs, its development must follow a structured process, and its output must be evaluated in terms of both software quality and educational value.

The input component consists of player commands from keyboard, mouse, touch, or joystick, Lua and Python programs written by the player, and the world seed, game assets, account data, and saved-world data. The process component covers the Godot engine, which runs gameplay, physics, user interface, audio, and rendering; the generation of worlds, caves, dungeons, mobs, and structures; the Lua and Python interpreters that execute the player's code; and the updating of automation, missions, inventory, and saves. The output component includes the rendered world, characters, animations, effects, and HUD; music, sound effects, notifications, and interaction feedback; code results, automation, error messages, and learning records; and the updated local saves and cloud-synced progress.



Figure 2. Conceptual framework

System Design Specification

SandEE is implemented in Godot 4.6 using GDScript for the game logic and scene composition. The project follows a modular, scene-based architecture in which the overworld, cave interiors, dungeon interiors, player, user interface, and programming tools are separate scenes or scripts coordinated by a central game instance. Four autoloads support the runtime: CrashLogger records unexpected errors, Global stores persistent state and shared services, Supabase provides optional online authentication and data access, and CloudSync manages two-way cloud-save synchronization. The main entry point is the startup mode scene, which restores a session when available and leads to the main menu.

The game uses a client-side architecture. All core gameplay, world generation, saving, and code execution run on the player's device, which allows the game to remain playable offline. Online features are optional and are accessed only when the player signs in. The language runtime is implemented as a Rust GDExtension: Lua is executed through the mlua library, while Python source is parsed with ruff_python_parser and transpiled to GDScript before execution. This design keeps the game engine independent of the language implementations while still providing real parsing, execution, and error reporting.

Functional Modules

Table 1. Functional modules and descriptions

Module	Description
World generation	Seeded terrain generation with plains, desert, snowy, and water areas; chunk-based structures, caves, and dungeon entrances.
Caves and dungeons	Cellular-automata caves, room-and-corridor pixie caves, and themed Lua/Python dungeons with knowledge and quiz altars.
Survival and crafting	Resource gathering, mining, smelting, crafting, building, farming, fishing, combat, and day/night survival.
Programming tools	In-game Lua and Python editors, syntax highlighting, code execution, error reporting, snippets, and the Programming Encyclopedia.
Automation	Automatica stations that mine, chop, build, deposit, and fight; Algorithmica stations that automate furnaces, crafting stations, and color crafters.
Missions and progression	Mission tracker, blue and yellow stars, ranks, completion badges, dungeon rewards, and leaderboards.
Accounts and saves	Optional username/password accounts, local JSON saves, autosave, cloud saves, conflict handling, and star totals through Supabase.
User interface	Main menu, HUD, inventory, hotbar, crafting and smelting panels, minimap, pause menu, settings, and mobile touch controls.

System Architecture

The system is organized into five layers. The presentation layer contains the startup, menu, HUD, inventory, crafting, code-editor, and mobile-control interfaces. The game

layer contains the overworld, caves, dungeons, player, combat, missions, and day/night cycle. The systems layer contains world generation, crafting and recipes, save/load, cloud sync, Automatica, automation hosts, waypoints, and the minimap. The language runtime layer contains the Rust GDExtension, the Lua runtime, the Python parser and transpiler, error reporting, and the code sandbox. The data layer contains Godot resource files, local JSON saves, and the Supabase services used for accounts, cloud storage, and leaderboards.

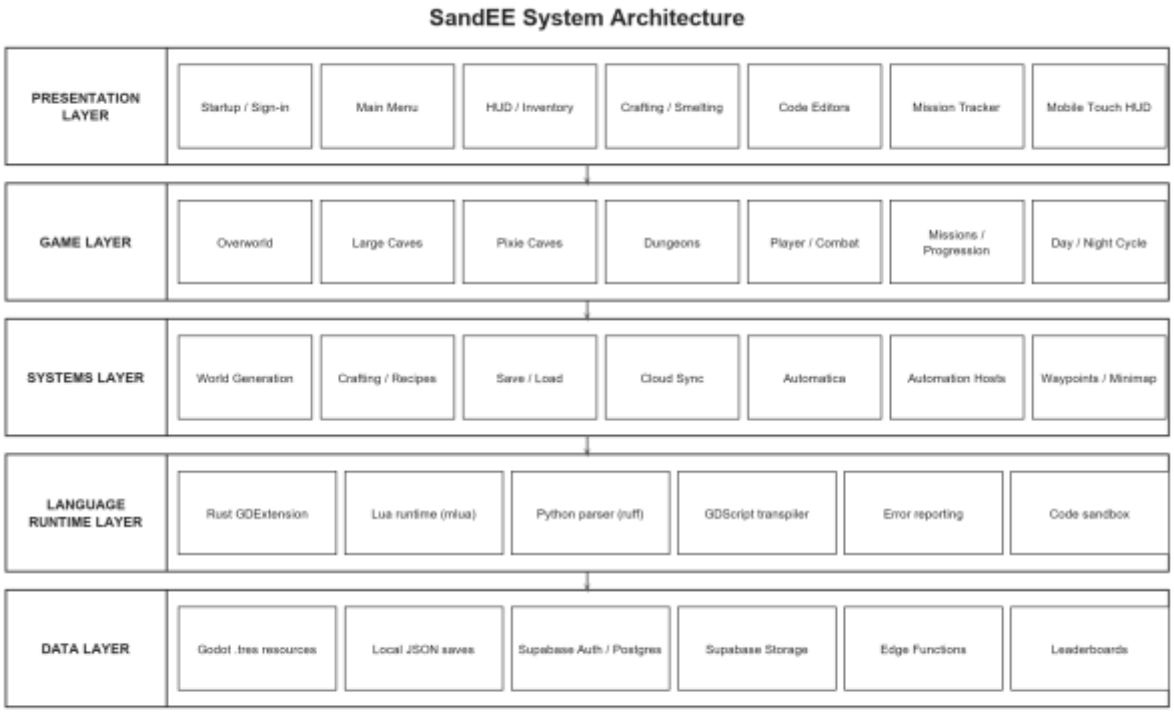


Figure 3. System architecture

World Generation and Environment

The overworld is generated from a random seed and divided into 25 by 25-tile chunks. Terrain is classified into water, plain, desert, and snowy biomes, and each chunk is populated with trees, grass, rocks, structures, mobs, and cave entrances according to seeded rules. Large caves use cellular-automata carving to produce organic caverns with stone and ore deposits. Pixie caves use room-and-corridor layouts with crystal ores and regenerating resources. Dungeons use a separate room-and-corridor generator that guarantees a knowledge room, a quiz room, an entrance room, and additional rooms for altars and chests. The world also includes a day/night cycle, ambient mob spawning, waypoints, and a minimap.

The world generator is deterministic: the same seed produces the same terrain, structures, cave entrances, and dungeon layouts. This is important for reproducibility, debugging, and fair play. The game also separates natural terrain from player-placed blocks, so the original world can be regenerated while player changes are restored from the save file.

Programming and Automation Module

The programming module is the educational core of SandEE. Players write Lua or Python in an in-game code editor that provides line numbers, syntax highlighting, snippets, and an output panel. Lua programs run through the embedded Lua runtime and can control Automatica. Python programs are parsed and transpiled to GDScript before execution, and can control Algorithmica stations such as furnaces and crafting stations. The editors report syntax and runtime errors back to the player, and a successful run produces visible changes in the game world. This creates a direct link between code, feedback, and gameplay.

Automatica can be renamed, given tools, assigned build materials, and switched into combat mode. They can chop trees, mine blocks, place blocks, deposit items, and fight nearby enemies. Algorithmica stations expose a Python API for loading inputs, adding fuel, smelting, crafting, and taking outputs. The Programming Encyclopedia and the Knowledge Altar provide reference material and unlockable explanations of programming concepts, while the Quiz Altar checks the player's understanding before a dungeon is cleared.

Save, Account, and Cloud System

Each world is stored in a local JSON save file associated with an account and a save slot. The game autosaves periodically and at important state changes such as entering a cave or dungeon, and it also saves when the player quits. When the player is signed in and online, CloudSync uploads the local save to Supabase Storage and records its metadata in the database. Cloud-only worlds can be downloaded into free local slots, and conflicts are resolved by revision, world identity, or manual selection. The online layer also supports profiles, star totals, and leaderboards, with row-level security enforced in the database.

The account and cloud features are optional. A player can create a local account and play offline, then switch to online mode later. This supports classrooms and devices with unreliable internet access while still allowing progress to be synchronized across devices.

User Interface and Cross-Platform Layer

The user interface is designed for both Windows and Android. On desktop, the player uses the keyboard and mouse for movement, interaction, building, and combat. On mobile, a virtual joystick, contextual action buttons, and tap-to-interact controls replace the mouse and keyboard. The HUD includes the hotbar, health and stamina bars, mission tracker, minimap, inventory, crafting panel, code editors, and pause menu. The game also uses a pixel-art visual style, responsive canvas scaling, and a consistent color palette across menus and in-game panels.

Diagrams

The following diagrams describe the flow of data, the structure of the data, the functional hierarchy, the account authentication process, and the main gameplay loops of SandEE. The data flow diagrams show how the player, the game system, local saves, and the optional cloud services exchange information. The entity relationship diagram shows the main data entities used by the save, mission, dungeon, and automation systems. The hierarchical chart shows the major modules of the game, and the flowcharts show the expected sequence of account access and player actions in the overworld and in dungeons.

Data Flow Diagram Level 0

Figure 4 presents the context diagram of SandEE. The player interacts with the game by issuing commands, writing code, and making menu choices. The game returns visual feedback, error messages, game state, and progress information. The game reads and writes local save files, and it exchanges authentication, cloud-save, and leaderboard data with Supabase when online features are enabled.

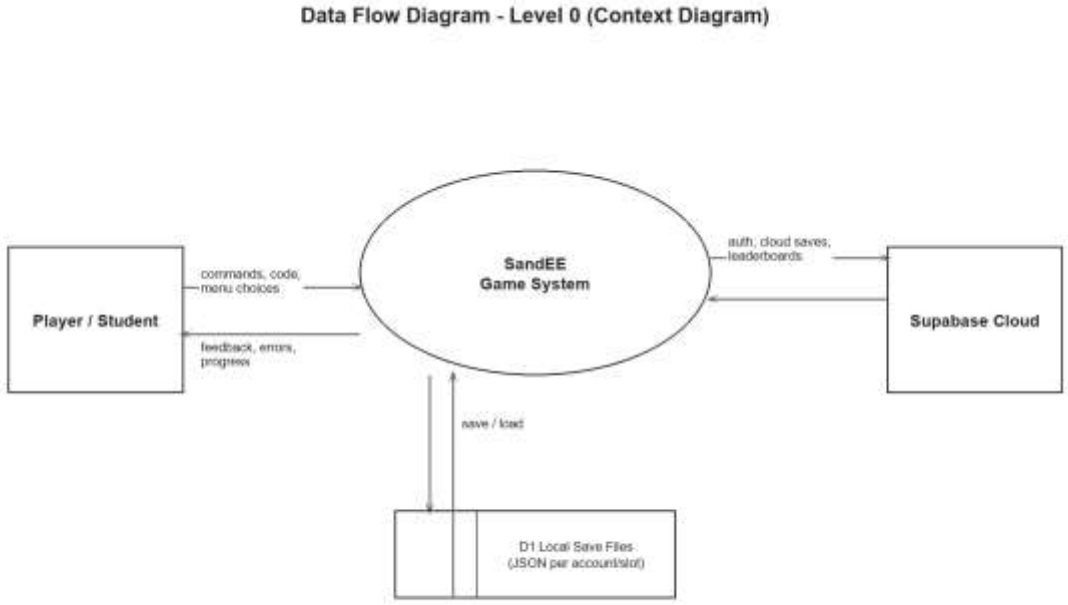


Figure 4. Data flow diagram level 0

Data Flow Diagram Level 1

Figure 5 decomposes the system into six processes. Account and save management handles sign-in, local saves, and cloud synchronization. World exploration manages terrain, caves, structures, and mobs. Gathering, crafting, and building manage resources, recipes, and placed blocks. Learning and code execution manage the editors, interpreters, and automation behavior. Mission and dungeon completion manage objectives, quizzes, rewards, and stars. Progress and leaderboard management maintain the player's profile, achievements, and ranking. The data stores hold the player profile, world save, mission progress, code programs, and game content.

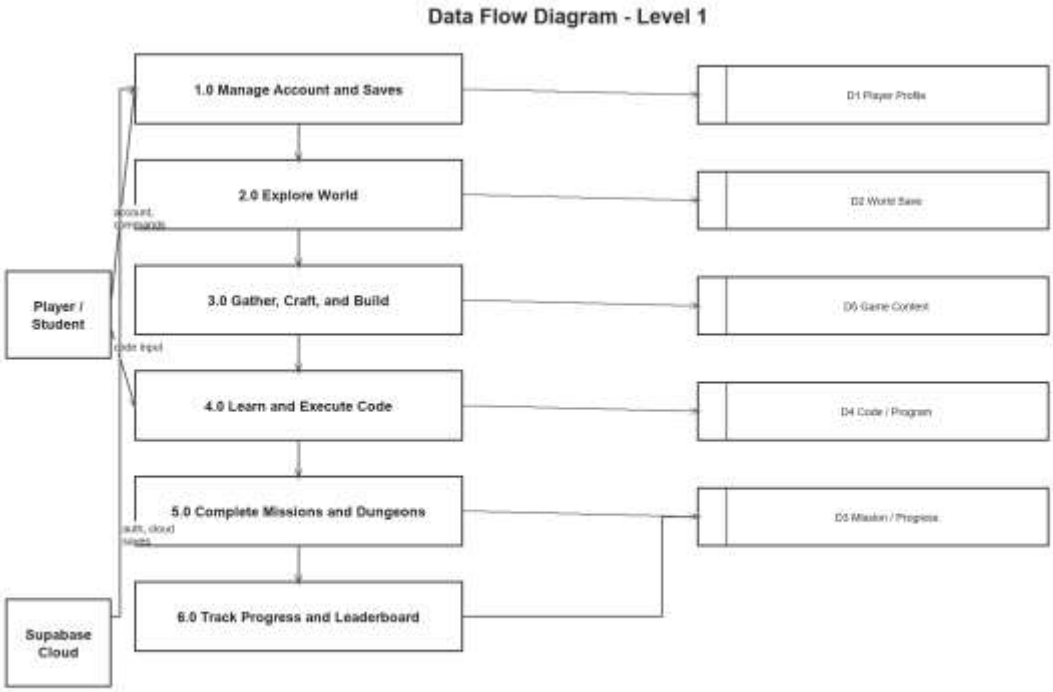


Figure 5. Data flow diagram level 1

Entity Relationship Diagram

Figure 6 shows the main entities used by the latest version of SandEE. A player can own multiple world saves, mission-progress records, cloud saves, bestiary entries, and encyclopedia unlocks. Each world save contains inventory slots, waypoints, Automatica, and automation stations. Missions generate dungeons, and each dungeon contains rooms with knowledge or quiz altars. Recipes reference the items they consume and produce, while Automatica and automation stations reference the code programs that control them. These relationships support the save, mission, dungeon, programming, and online systems.

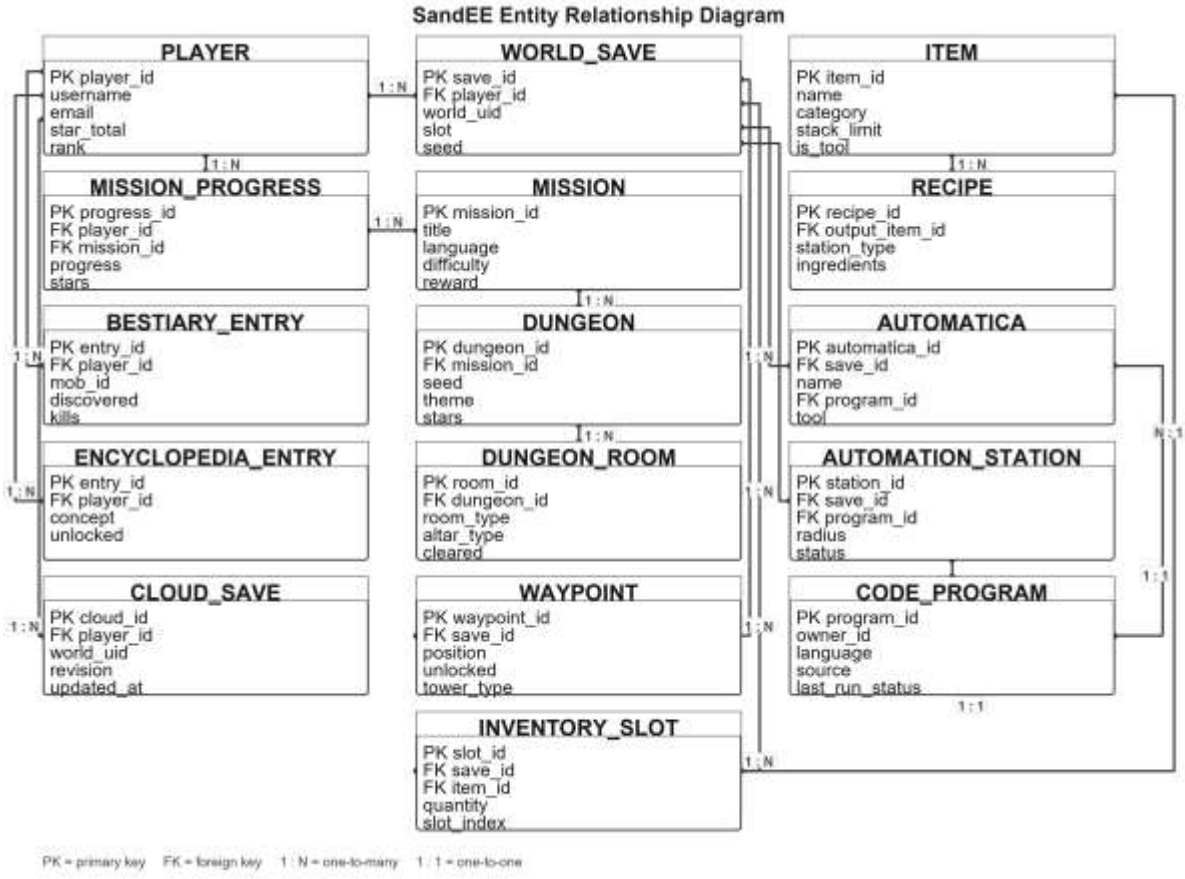


Figure 6. Entity relationship diagram

Hierarchical Chart

Figure 7 presents the functional hierarchy of SandEE. The main menu and account module provides access to worlds, leaderboards, cloud saves, and settings. The overworld module contains biomes, trees, water, caves, structures, and mobs. The cave module contains large caves, pixie caves, ores, and mining. The dungeon module contains the Lua and Python dungeons, knowledge and quiz altars. The programming module contains the Lua and Python editors, Automatica, Algorithmica, and the encyclopedia. The interface module contains the inventory, crafting, mission tracker, minimap, and mobile HUD. The online module contains authentication, cloud saves, leaderboards, and profiles.

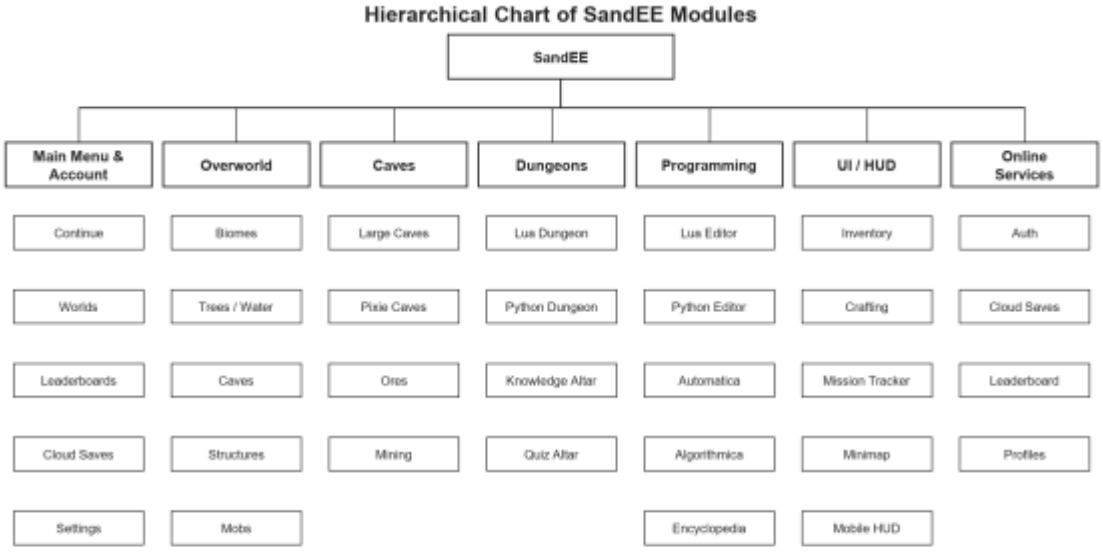


Figure 7. Hierarchical chart

Account Authentication Flowchart

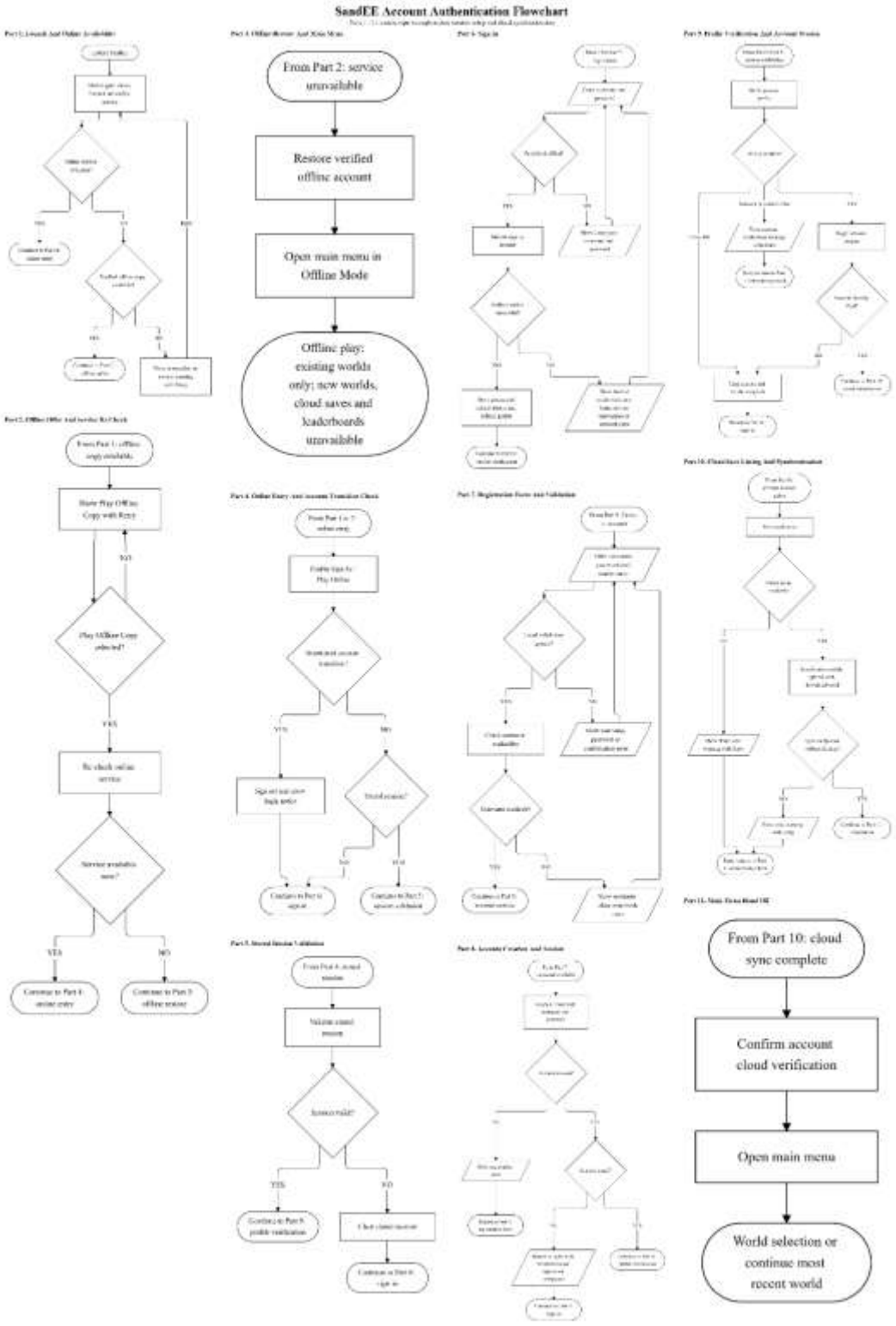


Figure 8. Account authentication flowchart

Overworld Gameplay Flowchart

From the main menu, the player loads or creates a world and follows the tutorial and guided objectives. The player explores the overworld, gathers resources, crafts items, builds structures, manages the inventory, and programs Automatica, Algorithmica stations, or Python programs. Missions, day/night encounters, caves, waypoints, and dungeons provide progression. Entering a dungeon leads to the Knowledge Altar, quiz and code challenges, and star rewards before returning to the overworld. The loop ends with autosave and cloud synchronization.

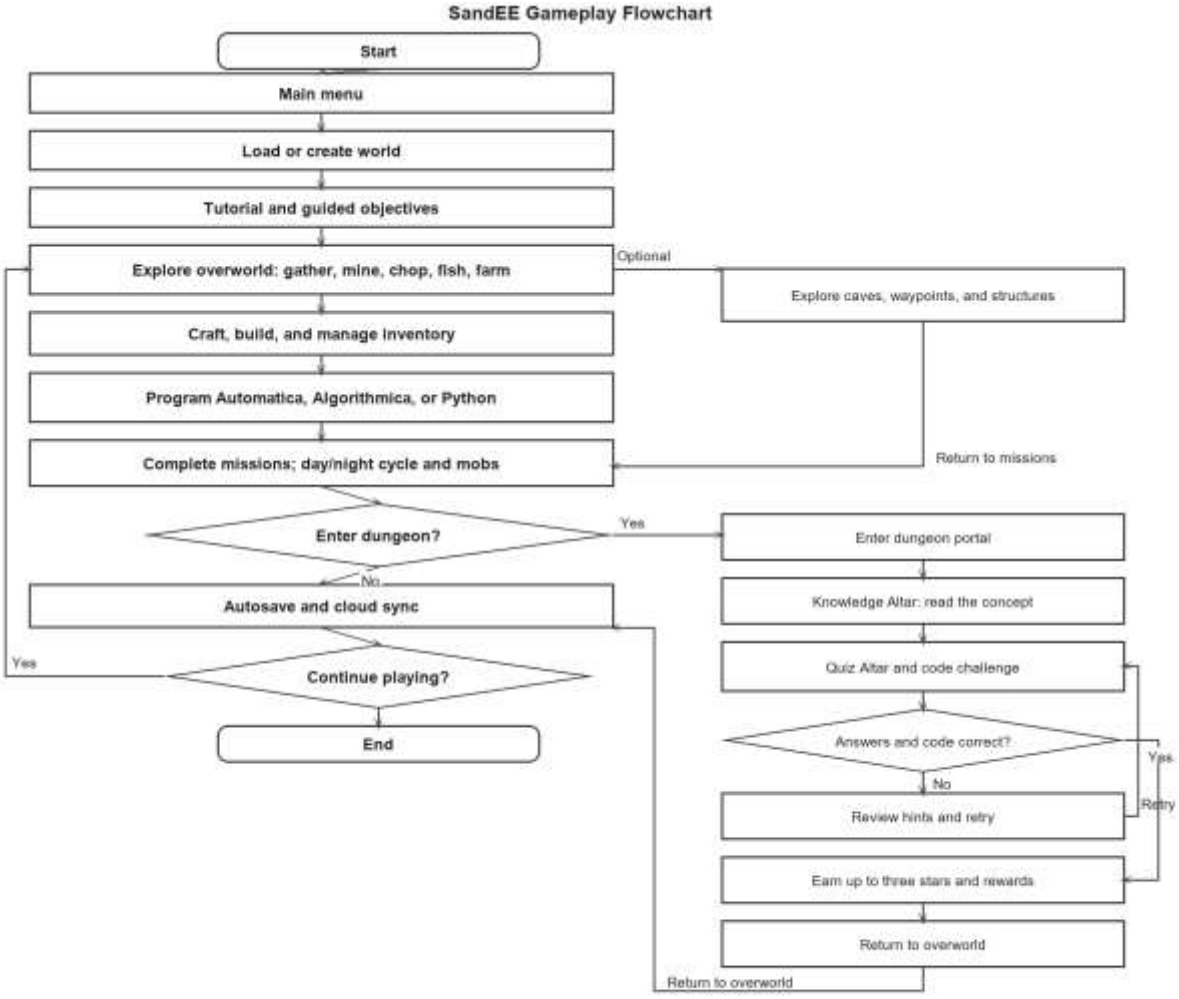


Figure 9. Updated gameplay flowchart

Dungeon Gameplay Flowchart

Figure 10 shows the flow inside a dungeon.

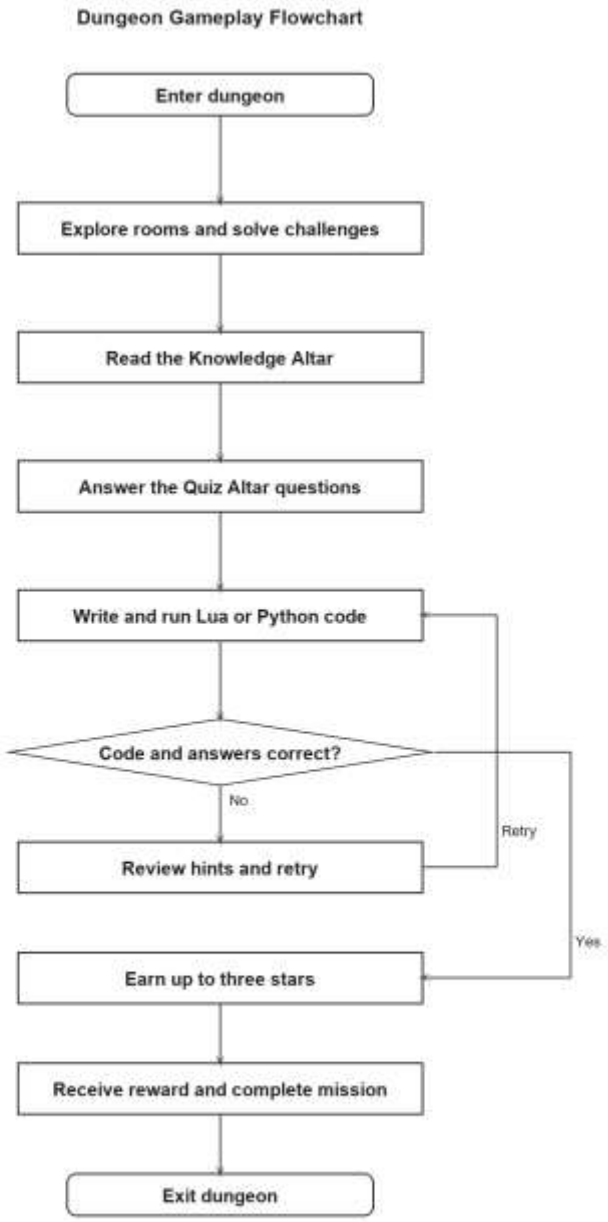


Figure 10. Dungeon gameplay flowchart

Hardware/Software

Table 2. Hardware

Devices	Specifications
Desktop	CPU: AMD Ryzen 5 5600G Motherboard: MSI PRO B550M-P GEN3 GPU: Radeon Vega 7 Integrated Graphics RAM: 16 GB Operating System: CachyOS (Arch Linux)
Laptop	CPU: i5-12500H Motherboard:AN515-58-50YE GPU: RTX 3050M Ram: 8x2 DDR4 2400mhz sodimm Operating System: Windows 10 Pro

Table 3. Software

Software	Purpose
Godot Engine	Game Engine
GDScript	Game Logic
Supabase	Game Database
Aseprite	Art Editing Software
Rust	Performance critical implementation
Lua	Main Scripting Language
Python	Sub Scripting Language
VS Codium	Chosen IDE.
GitHub	Version Control Software, Logger, File Storage

Calendar of Activities

During the first week of September, the developers began working on the initial draft of the study's introduction and finalized the research title. This phase focuses on identifying the study's purpose, significance, and scope while aligning the objectives with the project's intended outcome. The week concluded with the team reviewing the overall direction of the research to ensure that it addressed relevant academic and industry needs.

In the second week of September, the team revised the introduction to improve clarity and alignment with the research objectives. They also began drafting Chapter I, which detailed the background of the problem, statement of the problem, and significance of the study. The group collaborated closely to ensure that the content presented a strong foundation for the succeeding chapters.

The third week of September was dedicated to additional revisions and refinements of Chapter I. The developers reviewed the document to ensure consistency, coherence, and adherence to academic standards. Feedback from peers and advisers was considered to enhance the quality and flow of the content, preparing the chapter for final review.

During the fourth week of September, the developers conducted the survey to gather data from the target respondents. The collected information served as the basis for understanding user preferences and needs. In the same week, the team began drafting Chapter II, which included the review of related literature, studies, and systems relevant to the proposed project.

The first week of October was devoted to extensive research on literature, existing studies, and software systems related to game-based learning. The developers consolidated and analyzed this information to strengthen Chapter II. By the end of the week, Chapter II was completed and prepared for initial revision.

In the second week of October, the proponents focused on drafting the hardware and software selection section. This included identifying the tools, technologies, and development environments necessary for system implementation. The week also involved outlining the technical specifications and requirements for the proposed application.

During the third week of October, the team concentrated on revising and improving Chapter III, which presented the project's methodology. The developers refined the research design, data collection methods, and system development framework to ensure accuracy and alignment with research objectives. In the fourth week of October, the proponents carried out final revisions and grammar checks across all chapters to ensure consistency and correctness. The team also completed the initial design of the software user interface, marking the transition from the documentation phase to the beginning of system development.

Budgetary Estimate

Table 4. Budgetary Estimate

Quantity	Specifics	Approximate Cost
1	Aseprite	PHP 1, 275.29
	Total	PHP 1, 275.29

RESULTS AND DISCUSSIONS

Overview of the results

This chapter presents the results of the IT student and IT expert evaluation of SandEE. The IT student evaluation assessed the game's functional suitability, performance efficiency, and usability based on selected ISO 25010 criteria. A total of 70 IT students participated. The IT expert evaluation assessed the game's functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability.

The results are presented using tables. Frequency and percentage distributions are used to show the distribution of responses, while weighted means and standard deviations are used to summarize the ratings. The interpretation of the weighted means follows the four-point ISO 25010 quality scale presented in Table 5.

System Implementation

Figure 11 shows actual screenshots of SandEE during gameplay. The screenshots document the dungeon exploration area. These images provide evidence that the game was implemented and that the visual style, dungeon themes, and player interface are consistent with the design described in the methodology.



Figure 11. SandEE gameplay screenshot

Data Gathering Procedure

The student evaluation was conducted after the game was developed. The IT students were given a short demonstration of the game's main features, including the main menu, world exploration, crafting, building, the Lua and Python editors, Automatica, Algorithmica stations, and the dungeon challenges. They were then asked to evaluate the game using a four-point ISO 25010 quality scale questionnaire. The evaluation involved 70 IT students and three IT experts. The IT experts were selected based on their background in software development, programming education, or game development.

For the IT students, the instrument covered functional suitability, performance efficiency, and usability. For the IT experts, it covered functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability. Each criterion was represented by several statements, and the respondents rated each statement from 1 (Poorly Functional/Efficient/Usable) to 4 (Very Functional/Efficient/Usable), depending on the ISO 25010 criterion. The responses were tallied, and the weighted mean and standard deviation were computed for each criterion.

Statistical Treatment of Data

The data were analyzed using frequency, percentage, weighted mean, and standard deviation. Frequency refers to the number of respondents who selected a particular response, while percentage is the frequency divided by the total number of respondents multiplied by 100. The weighted mean is computed by multiplying each response value by its frequency, summing the products, and dividing the sum by the total number of responses. The standard deviation measures the spread of the responses around the mean; a small standard deviation indicates that the respondents' ratings were consistent.

The weighted mean is interpreted using the four-point ISO 25010 quality scale in Table 5. A mean within 3.26–4.00 is interpreted as Very Functional, Very Efficient, or Very Usable; 2.51–3.25 as Functional, Efficient, or Usable; 1.76–2.50 as Moderately Functional, Moderately Efficient, or Moderately Usable; and 1.00–1.75 as Poorly Functional, Poorly Efficient, or Poorly Usable.

Table 5. Mean range interpretation legend

Mean Range	Interpretation
3.26 – 4.00	Very High / Very Functional / Very Efficient / Very Usable
2.51 – 3.25	High / Functional / Efficient / Usable
1.76 – 2.50	Low / Moderately Functional / Moderately Efficient / Moderately Usable
1.00 – 1.75	Very Low / Poorly Functional / Poorly Efficient / Poorly Usable

User Feedback

This section presents the results of the IT student evaluation of SandEE. A total of 70 IT students participated in the evaluation and rated the game using a four-point ISO 25010 quality scale. The responses were tallied, and the weighted mean and standard deviation were computed for each item. The results are presented per indicator, followed by a summary by criterion.

Table 6. Functional completeness - Degree to which the set of functions covers all the specified tasks and user objectives

Metric	Poorly Functional	Moderately Functional	Functional	Very Functional
Average Frequency (%)	0	0	37.14%	62.86%
Rank	1	2	3	4

The results show that 62.86% of the IT students rated SandEE as Very Functional for functional completeness, while 37.14% rated it Functional. No respondents rated it Moderately Functional or Poorly Functional. This indicates that most IT students perceive the system as functional and that it covers all the specified tasks and user objectives.

Table 7. Functional correctness - Degree to which a product or system provides the correct results with the needed degree of precision

Metric	Poorly Functional	Moderately Functional	Functional	Very Functional
Average Frequency (%)	0	0	34.29%	65.71%
Rank	1	2	3	4

The results show that 65.71% of the IT students rated SandEE as Very Functional for functional correctness, while 34.29% rated it Functional. No respondents rated it Moderately Functional or Poorly Functional. This indicates that most IT students perceive the system as functional and that it provides correct results with the needed degree of precision.

Table 8. Functional appropriateness - Degree to which the functions facilitate the accomplishment of specified tasks and objectives

Metric	Poorly Functional	Moderately Functional	Functional	Very Functional
Average Frequency (%)	0	0	40.00%	60.00%
Rank	1	2	3	4

The results show that 60.00% of the IT students rated SandEE as Very Functional for functional appropriateness, while 40.00% rated it Functional. No respondents rated it Moderately Functional or Poorly Functional. This indicates that most IT students perceive the system as functional and that its functions facilitate the accomplishment of specified tasks and objectives.

Table 9. Time behaviour - Degree to which the response and processing times and throughput rates of system, when performing functions, meet requirements

Metric	Poorly Efficient	Moderately Efficient	Efficient	Very Efficient
Average Frequency (%)	0	0	45.71%	54.29%
Rank	1	2	3	4

The results show that 54.29% of the IT students rated SandEE as Very Efficient for time behaviour, while 45.71% rated it Efficient. No respondents rated it Moderately Efficient or Poorly Efficient. This indicates that most IT students perceive the system as efficient and that its response and processing times and throughput rates meet the requirements.

Table 10. Resource utilization - Degree to which the amounts and types of resources used by a system, when performing its functions, meet requirements

Metric	Poorly Efficient	Moderately Efficient	Efficient	Very Efficient
Average Frequency (%)	0	0	48.57%	51.43%
Rank	1	2	3	4

The results show that 51.43% of the IT students rated SandEE as Very Efficient for resource utilization, while 48.57% rated it Efficient. No respondents rated it Moderately Efficient or Poorly Efficient. This indicates that most IT students perceive the system as efficient and that it uses appropriate amounts and types of resources when performing its functions.

Table 11. Capacity - Degree to which the maximum limits of a product or system parameter meet requirements

Metric	Poorly Efficient	Moderately Efficient	Efficient	Very Efficient
Average Frequency (%)	0	0	42.86%	57.14%
Rank	1	2	3	4

The results show that 57.14% of the IT students rated SandEE as Very Efficient for capacity, while 42.86% rated it Efficient. No respondents rated it Moderately Efficient or Poorly Efficient. This indicates that most IT students perceive the system as efficient and that its maximum limits meet the requirements.

Table 12. Appropriateness recognizability - Degree to which users can recognize whether a system is appropriate for their needs

Metric	Poorly Usable	Moderately Usable	Usable	Very Usable
Average Frequency (%)	0	0	35.71%	64.29%
Rank	1	2	3	4

The results show that 64.29% of the IT students rated SandEE as Very Usable for appropriateness recognizability, while 35.71% rated it Usable. No respondents rated it Moderately Usable or Poorly Usable. This indicates that most IT students perceive the system as usable and that users can recognize whether SandEE is appropriate for their needs.

Table 13. Learnability - Degree to which a system can be used by specified users to achieve specified goals of learning to use the product or system with effectiveness, efficiency, freedom from risk and satisfaction in a specified context of use

Metric	Poorly Usable	Moderately Usable	Usable	Very Usable
Average Frequency (%)	0	0	32.86%	67.14%
Rank	1	2	3	4

The results show that 67.14% of the IT students rated SandEE as Very Usable for learnability, while 32.86% rated it Usable. No respondents rated it Moderately Usable or Poorly Usable. This indicates that most IT students perceive the system as usable and that it can be learned effectively, efficiently, and with satisfaction.

Table 14. Operability - Degree to which a system has attributes that make it easy to operate and control

Metric	Poorly Usable	Moderately Usable	Usable	Very Usable
Average Frequency (%)	0	0	38.57%	61.43%
Rank	1	2	3	4

The results show that 61.43% of the IT students rated SandEE as Very Usable for operability, while 38.57% rated it Usable. No respondents rated it Moderately Usable or Poorly Usable. This indicates that most IT students perceive the system as usable and that it is easy to operate and control.

Table 15. User error protection - Degree to which a system protects users against making errors

Metric	Poorly Usable	Moderately Usable	Usable	Very Usable
Average Frequency (%)	0	0	44.29%	55.71%
Rank	1	2	3	4

The results show that 55.71% of the IT students rated SandEE as Very Usable for user error protection, while 44.29% rated it Usable. No respondents rated it Moderately Usable or Poorly Usable. This indicates that most IT students perceive the system as usable and that it protects users against making errors.

Table 16. User interface aesthetics - Degree to which a user interface enables pleasing and satisfying interaction for the user

Metric	Poorly Usable	Moderately Usable	Usable	Very Usable
Average Frequency (%)	0	0	31.43%	68.57%
Rank	1	2	3	4

The results show that 68.57% of the IT students rated SandEE as Very Usable for user interface aesthetics, while 31.43% rated it Usable. No respondents rated it Moderately Usable or Poorly Usable. This indicates that most IT students perceive the system as usable and that its user interface enables pleasing and satisfying interaction.

Table 17. Accessibility - Degree to which a system can be used by people with the widest range of characteristics and capabilities to achieve a specified goal in a specified context of use

Metric	Poorly Usable	Moderately Usable	Usable	Very Usable
Average Frequency (%)	0	0	41.43%	58.57%
Rank	1	2	3	4

The results show that 58.57% of the IT students rated SandEE as Very Usable for accessibility, while 41.43% rated it Usable. No respondents rated it Moderately Usable or Poorly Usable. This indicates that most IT students perceive the system as usable

and that it can be used by people with a wide range of characteristics and capabilities.

Summary of IT Student Evaluation Results

Table 18. Summary of IT student evaluation results

Criterion	Weighted Mean	Standard Deviation	Interpretation
Functional Suitability	3.63	0.48	Very Functional
Performance Efficiency	3.54	0.50	Very Efficient
Usability	3.63	0.48	Very Usable
Overall	3.61	0.49	Very High

The summary shows that functional suitability obtained a weighted mean of 3.63 (SD = 0.48), interpreted as Very Functional; performance efficiency obtained 3.54 (SD = 0.50), interpreted as Very Efficient; and usability obtained 3.63 (SD = 0.48), interpreted as Very Usable. All three criteria fall within the 3.26–4.00 range. The overall weighted mean of 3.61 (SD = 0.49) is interpreted as Very High, indicating that the IT students rated SandEE favorably across the evaluated ISO 25010 characteristics.

Expert Feedback

This section presents the results of the IT expert evaluation of SandEE. Three IT experts rated the game using the eight ISO 25010 characteristics: functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability. Each criterion was measured through several indicators. Tables 19 to 26 present the frequency distribution for each indicator, while the weighted mean, standard deviation, and interpretation are discussed in the paragraph below each table. Table 27 summarizes the results by criterion. For the frequency columns, values are counts with percentages in parentheses (n = 3); a rating of 1 is the lowest and 4 is the highest.

Table 19. Functional Suitability - IT expert evaluation

Indicator	1	2	3	4
Functional completeness	0 (0.00%)	0 (0.00%)	2 (66.67%)	1 (33.33%)
Functional correctness	0 (0.00%)	2 (66.67%)	1 (33.33%)	0 (0.00%)
Functional appropriateness	0 (0.00%)	0 (0.00%)	2 (66.67%)	1 (33.33%)

The IT experts rated functional suitability with an overall mean of 3.00 (SD = 0.71), interpreted as Functional. Functional completeness obtained 3.33 (SD = 0.58) and functional appropriateness 3.33 (SD = 0.58), both Very Functional, while functional correctness obtained 2.33 (SD = 0.58), interpreted as Moderately Functional. This indicates that the experts found the game's functions complete and appropriate for the intended tasks, but they were less convinced that all outputs were consistently correct and precise.

Table 20. Performance Efficiency - IT expert evaluation

Indicator	1	2	3	4
Time behaviour	0 (0.00%)	0 (0.00%)	3 (100.00%)	0 (0.00%)
Resource utilization	0 (0.00%)	0 (0.00%)	0 (0.00%)	3 (100.00%)
Capacity	0 (0.00%)	0 (0.00%)	1 (33.33%)	2 (66.67%)

Performance efficiency obtained 3.56 (SD = 0.53), interpreted as Very Efficient. Resource utilization was rated 4.00 (SD = 0.00) and capacity 3.67 (SD = 0.58), both Very Efficient, while time behaviour obtained 3.00 (SD = 0.00), interpreted as Efficient. The experts therefore found the game's resource use and capacity strong, with response and processing time adequate but not the strongest area.

Table 21. Compatibility - IT expert evaluation

Indicator	1	2	3	4
Co-existence	0 (0.00%)	0 (0.00%)	0 (0.00%)	3 (100.00%)
Interoperability	0 (0.00%)	0 (0.00%)	1 (33.33%)	2 (66.67%)

Compatibility obtained 3.83 (SD = 0.41), interpreted as Very Compatible. Co-existence was rated 4.00 (SD = 0.00) and interoperability 3.67 (SD = 0.58), both Very Compatible. This shows that the experts saw the game as able to operate alongside other software and exchange information with other systems without detrimental impact.

Table 22. Usability - IT expert evaluation

Indicator	1	2	3	4
Appropriateness recognizability	0 (0.00%)	0 (0.00%)	3 (100.00%)	0 (0.00%)
Learnability	0 (0.00%)	0 (0.00%)	0 (0.00%)	3 (100.00%)
Operability	0 (0.00%)	1 (33.33%)	2 (66.67%)	0 (0.00%)
User error protection	0 (0.00%)	0 (0.00%)	0 (0.00%)	3 (100.00%)
User interface aesthetics	0 (0.00%)	0 (0.00%)	3 (100.00%)	0 (0.00%)
Accessibility	0 (0.00%)	0 (0.00%)	3 (100.00%)	0 (0.00%)

Usability obtained 3.28 (SD = 0.57), interpreted as Very Usable. Learnability and user error protection both received 4.00 (SD = 0.00), while appropriateness recognizability, user interface aesthetics, and accessibility each received 3.00 (SD = 0.00), interpreted as Usable; operability obtained 2.67 (SD = 0.58), also Usable. The experts found the game easy to learn and forgiving of errors, but they saw room to improve operability and the clarity of some interface elements.

Table 23. Reliability - IT expert evaluation

Indicator	1	2	3	4
Maturity	0 (0.00%)	0 (0.00%)	3 (100.00%)	0 (0.00%)
Availability	0 (0.00%)	0 (0.00%)	3 (100.00%)	0 (0.00%)
Fault tolerance	0 (0.00%)	0 (0.00%)	1 (33.33%)	2 (66.67%)
Recoverability	0 (0.00%)	0 (0.00%)	0 (0.00%)	3 (100.00%)

Reliability obtained 3.42 (SD = 0.51), interpreted as Very Reliable. Recoverability received 4.00 (SD = 0.00) and fault tolerance 3.67 (SD = 0.58), both Very Reliable, while maturity and availability each received 3.00 (SD = 0.00), interpreted as Reliable. The experts considered the game stable and recoverable, with normal operation and availability rated positively but slightly lower than fault tolerance and recovery.

Table 24. Security - IT expert evaluation

Indicator	1	2	3	4
Confidentiality	0 (0.00%)	2 (66.67%)	1 (33.33%)	0 (0.00%)
Integrity	0 (0.00%)	2 (66.67%)	1 (33.33%)	0 (0.00%)
Non-repudiation	0 (0.00%)	0 (0.00%)	3 (100.00%)	0 (0.00%)
Accountability	0 (0.00%)	0 (0.00%)	3 (100.00%)	0 (0.00%)
Authenticity	0 (0.00%)	0 (0.00%)	3 (100.00%)	0 (0.00%)

Security obtained 2.73 (SD = 0.46), interpreted as Secure. Non-repudiation, accountability, and authenticity each received 3.00 (SD = 0.00), while confidentiality and integrity each received 2.33 (SD = 0.58), interpreted as Moderately Secure. This indicates that the experts viewed the game's security mechanisms as adequate but identified confidentiality and data integrity as the weakest aspects of the system.

Table 25. Maintainability - IT expert evaluation

Indicator	1	2	3	4
Modularity	0 (0.00%)	0 (0.00%)	2 (66.67%)	1 (33.33%)
Reusability	0 (0.00%)	0 (0.00%)	3 (100.00%)	0 (0.00%)
Analysability	0 (0.00%)	0 (0.00%)	3 (100.00%)	0 (0.00%)
Modifiability	0 (0.00%)	0 (0.00%)	2 (66.67%)	1 (33.33%)
Testability	0 (0.00%)	0 (0.00%)	2 (66.67%)	1 (33.33%)

Maintainability obtained 3.20 (SD = 0.41), interpreted as Maintainable. Modularity, modifiability, and testability each received 3.33 (SD = 0.58), interpreted as Very Maintainable, while reusability and analysability each received 3.00 (SD = 0.00), interpreted as Maintainable. The experts found the system modular and testable, with reuse and analysis remaining areas for further improvement.

Table 26. Portability - IT expert evaluation

Indicator	1	2	3	4
Adaptability	0 (0.00%)	0 (0.00%)	0 (0.00%)	3 (100.00%)
Installability	0 (0.00%)	0 (0.00%)	0 (0.00%)	3 (100.00%)
Replaceability	0 (0.00%)	0 (0.00%)	1 (33.33%)	2 (66.67%)

Portability obtained 3.89 (SD = 0.33), interpreted as Very Portable. Adaptability and installability each received 4.00 (SD = 0.00), and replaceability received 3.67 (SD = 0.58), all interpreted as Very Portable. The experts therefore rated the game's ability to run across environments and be installed or replaced very favorably.

Table 27. Summary of IT expert evaluation results

Criterion	Weighted Mean	Standard Deviation	Interpretation
Functional Suitability	3.00	0.71	Functional
Performance Efficiency	3.56	0.53	Very Efficient
Compatibility	3.83	0.41	Very Compatible
Usability	3.28	0.57	Very Usable
Reliability	3.42	0.51	Very Reliable
Security	2.73	0.46	Secure
Maintainability	3.20	0.41	Maintainable
Portability	3.89	0.33	Very Portable
Overall	3.29	0.60	Very High

The summary of the IT expert evaluation shows that portability (3.89, SD = 0.33), compatibility (3.83, SD = 0.41), and performance efficiency (3.56, SD = 0.53) were the highest-rated criteria, all interpreted as Very Portable, Very Compatible, and Very Efficient. Functional suitability (3.00, SD = 0.71), maintainability (3.20, SD = 0.41), and security (2.73, SD = 0.46) received the lowest criterion means, although functional suitability and maintainability were still interpreted as Functional and Maintainable, and security as Secure. The overall weighted mean of 3.29 (SD = 0.60) is interpreted as Very High, indicating that the IT experts rated SandEE favorably across the eight ISO 25010 characteristics.

The three IT experts also provided qualitative feedback on the game's design, functionality, and educational value. They commended its interesting concept, engaging gameplay, appealing art style, and soundtrack. They noted the modular structure of the system, the use of genuine Lua and Python execution, and the consistency of the pixel-art interface. They observed that the game provides a clear progression from exploration to programming-based automation, which makes it suitable for beginner learners. They also noted that some gameplay elements can be confusing, particularly after the tutorial, and that mission text can feel overwhelming for first-time players.

The experts recommended lowering the default sound volume, adding full-screen support on PC, improving the tutorial and object-interaction instructions, providing clearer post-tutorial objectives, reducing mission text overload, and adding more story or lore elements. These recommendations are reflected in the recommendations section.

CONCLUSION

Summary

This study developed and evaluated SandEE, an educational multi-platform 2D open-world survival crafting game designed to supplement the learning of Lua and Python among Grade 11 and 12 ICT and STEM students of Yverdon de Pestalozzi School Inc. The game combines procedural world generation, survival and crafting, mission-based progression, themed Lua and Python dungeons, knowledge and quiz altars, a programming encyclopedia, and embedded language interpreters that allow player-written programs to control Automatica and Algorithmica stations. It runs on Windows and Android, supports offline play, and offers optional online accounts, cloud saves, and leaderboards. The study used the Agile methodology and evaluated the completed system through an ISO 25010-based survey of 70 IT students and a quantitative and qualitative evaluation by three IT experts.

The IT student evaluation produced an overall weighted mean of 3.61 (SD = 0.49), interpreted as Very High. Functional suitability obtained 3.63 (SD = 0.48), performance efficiency 3.54 (SD = 0.50), and usability 3.63 (SD = 0.48), all within the Very Functional, Very Efficient, and Very Usable ranges. Every indicator received majority ratings of Very Functional, Very Efficient, or Very Usable, and no respondent rated any indicator as Poorly or Moderately. The IT expert evaluation produced an overall weighted mean of 3.29 (SD = 0.60), also interpreted as Very High, with portability, compatibility, and performance efficiency as the highest-rated criteria. These results show that SandEE met its functional requirements and was received favorably in terms of performance and usability.

Conclusions

SandEE was found to be Very Functional, with a weighted mean of 3.63 (SD = 0.48). Functional completeness was rated Very Functional by 62.86% of the IT students, functional correctness by 65.71%, and functional appropriateness by 60.00%; the remaining respondents rated these indicators Functional, and none rated them Moderately Functional or Poorly Functional. These results confirm that the game covers the specified tasks and user objectives, produces correct results with the needed precision, and provides functions that directly facilitate the intended learning activities. The combination of exploration, resource gathering, crafting, building, missions, dungeons, and programming challenges gives players a coherent set of activities that support the study's objectives rather than isolated features.

Performance efficiency obtained 3.54 (SD = 0.50), interpreted as Very Efficient. Time behaviour was rated Very Efficient by 54.29% of the respondents, resource utilization by 51.43%, and capacity by 57.14%, with the rest rating these indicators Efficient. Although all performance indicators were positive, the relatively lower rating for resource utilization shows that memory use, asset loading, and performance on less powerful devices remain the most practical areas for improvement. Usability obtained 3.63 (SD = 0.48), interpreted as Very Usable. Appropriateness recognizability was rated Very Usable by 64.29%, learnability by 67.14%, operability by 61.43%, user error protection by 55.71%, user interface aesthetics by 68.57%, and accessibility by 58.57%. These findings indicate that players can recognize the purpose of the game, learn its controls and programming workflow, operate its interface, and recover from errors, while the comparatively lower scores for error protection and accessibility identify where further usability work is most needed.

The IT expert evaluation produced an overall weighted mean of 3.29 (SD = 0.60), interpreted as Very High. Portability (3.89, SD = 0.33), compatibility (3.83, SD = 0.41), and performance efficiency (3.56, SD = 0.53) were the highest-rated criteria, while security (2.73, SD = 0.46), functional suitability (3.00, SD = 0.71), and maintainability (3.20, SD = 0.41) received the lowest criterion means. The experts commended the modular structure of the system, the use of genuine Lua and Python execution rather than simulated outputs, and the consistency of the pixel-art interface. They also observed that the game presents a clear progression from simple exploration and resource gathering to mission-based challenges and programming-driven automation, which makes it appropriate for beginner learners. Their feedback confirmed that the functional, instructional, and visual components of SandEE work together as an integrated educational game and that the system performs its intended functions accurately and consistently during evaluation.

Taken together, the student and expert evaluations show that SandEE achieved its development goals. The student evaluation produced an overall weighted mean of 3.61 (SD = 0.49), and the expert evaluation produced 3.29 (SD = 0.60); both are interpreted as Very High. The narrow spread of student responses indicates that the positive ratings were consistent across the respondents.

The results establish that the game is functional, efficient, and usable, and that it can serve as a supplementary tool for programming education by giving students a contextual environment in which to practice Lua and Python. The findings also show that the game does not replace classroom instruction or the formal curriculum; instead, it reinforces both by allowing students to apply programming concepts, observe the results of their code, and learn from errors in a controlled and engaging setting.

Recommendations

Drawing on the results and conclusions of this study, the suggestions below are intended to guide the development of future game-based learning systems. They are grounded in the IT experts' feedback and in the criteria that received the lowest ratings during the evaluation.

The most valuable direction for future systems lies in introducing multiplayer features, since learning is often stronger when students work together. Cooperative missions would let learners share ideas and debug Lua and Python code as a team. Clearer objectives and missions that build on one another would improve progression and guidance, while contextual prompts for interactions, a lower default sound volume, a full-screen mode, and an adjustable interface scale on PC would make the game more comfortable to use.

Future systems should also apply artificial intelligence inside the gameplay itself, such as an AI guide that gives hints, explains errors in plain language, and adapts mission difficulty to each learner.

Finally, future systems should be sustained through continued usability testing with beginner learners, performance profiling on low-end Android devices, and regular updates based on student and expert feedback, while keeping offline play for classrooms with limited connectivity.

By pursuing these recommendations, future developers and researchers can build on the foundation of this study and create game-based learning systems that are more engaging, more accessible, and more effective in helping students learn programming.

REFERENCES

Pohl, M., Rester, M., Judmaier, P. (2009). Interactive Game Based Learning: Advantages and Disadvantages. In: Stephanidis, C. (eds) Universal Access in Human-Computer Interaction. Applications and Services. UAHCI 2009. Lecture Notes in Computer Science, vol 5616. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-02713-0_10

D. Zhao, C. H. Muntean, A. E. Chis, G. Rozinaj and G. -M. Muntean, "Game-Based Learning: Enhancing Student Experience, Knowledge Gain, and Usability in Higher Education Programming Courses," in IEEE Transactions on Education, vol. 65, no. 4, pp. 502-513, Nov. 2022, doi: 10.1109/TE.2021.3136914.

Lampropoulos, G. (2023). Educational benefits of digital game-based learning: K-12 teachers' perspectives and attitudes. Advances in Mobile Learning Educational Research, 3(2), 805-817. https://doi.org/10.25082/AMLER.2023.02.008

Becker, B. A., Denny, P., Finnie-Ansley, J., Luxton-Reilly, A., Prather, J., & Santos, E. A. (2023). Programming is hard—or at least it used to be: Educational opportunities and challenges of AI code generation. In Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE 2023) (pp. 500–506). Association for Computing Machinery. https://doi.org/10.1145/3545945.3569759

Brumbaugh, Z., & Leithner, M. (2023). The Lua programming language. In Programming languages: Concepts and implementations (pp. 45–60). Springer. https://doi.org/10.1007/979-8-8688-0060-3_3

Crudu, V., & MoldStud Research Team. (2024, September 27). Are Lua developers in high demand in the job market? MoldStud. https://moldstud.com/articles/p-are-lua-developers-in-high-demand-in-the-job-market

Ierusalimschy, R., Figueiredo, L. H. de, & Celes, W. (2025). The evolution of Lua, continued. Journal of Computer Languages, 83, 101326. https://doi.org/10.1016/j.cola.2025.101326

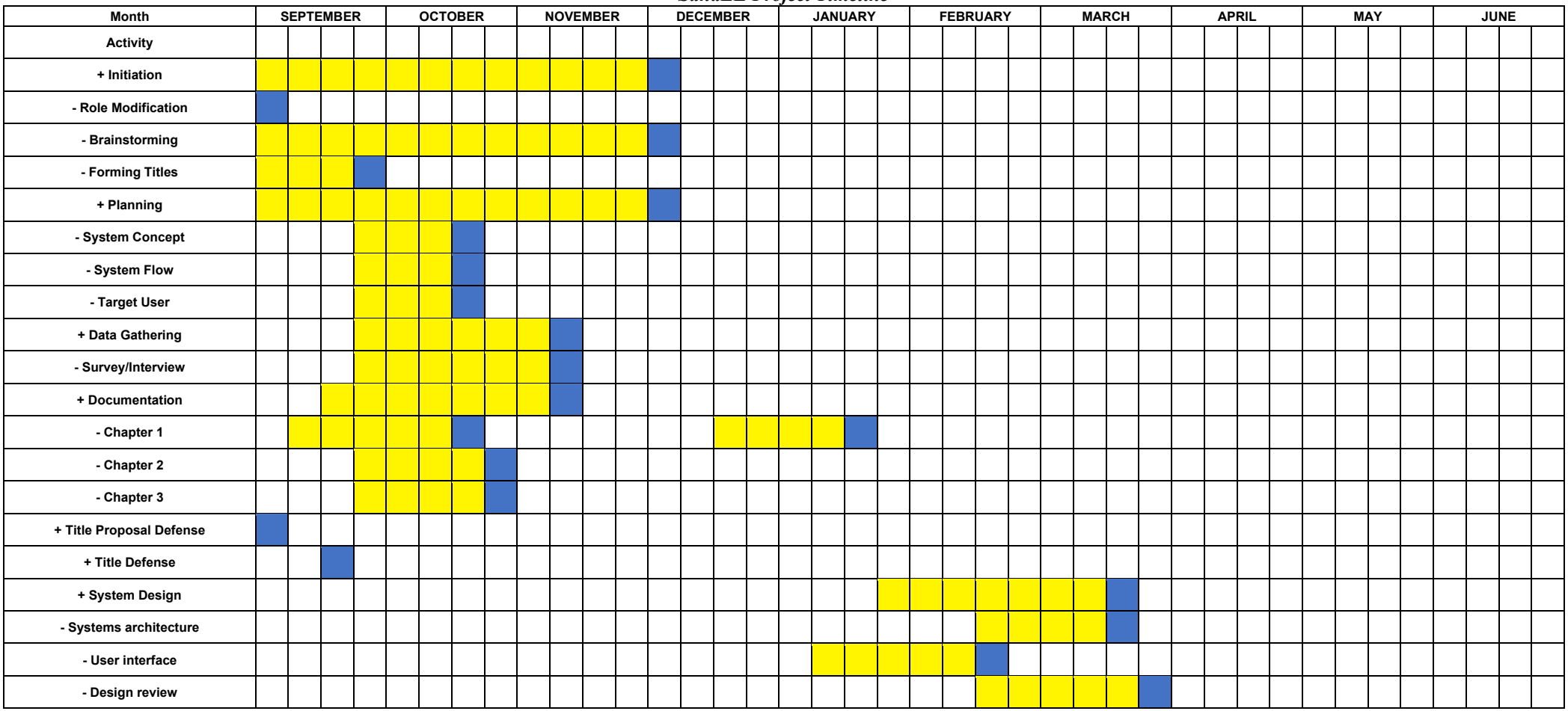
Lin, H.-C. K., Tseng, C.-H., & Chen, N.-S. (2025). Enhancing programming education: The impact of AI-based pedagogical agents on student self-efficacy, engagement, and learning outcomes. Educational Technology & Society, 28(2), 279–294. https://www.jstor.org/stable/48827987

Python, W. (2021). Python: Python releases for Windows. Python Software Foundation. https://www.python.org/downloads/windows/

Viduka, D., Kraguljac, V., & Ličina, B. (2021). A comparative analysis of the benefits of Python and Java for beginners. Quaestus, (19), 318–327.

APPENDIX A. GANTT CHART

SandEE Project Timeline



Month	JULY				AUGUST				SEPTEMBER				OCTOBER				NOVEMBER			
Activity																				
- Chapter 4																				
- Chapter 5																				
+ Development and Iteration																				
- Implementation																				
- Coding																				
+ Testing																				
- Bug Hunting																				
+ Deployment																				
- System deployment																				
+ Review and Feedback																				
- User Feedback Collection																				
- Bug fixes																				
+ Defense																				
- Mock Defense																				
- Re-Defense																				

APPENDIX B. ACTUAL THESIS EXPENSES

Table 28 presents the actual expenses incurred during the development and documentation of the study. The approximate costs were based on the budgetary estimate, while the actual costs reflect the final expenditures.

Table 28. Actual thesis expenses

Quantity	Specifics	Approximate Cost	Actual Cost
1	Aseprite	PHP 1, 279.29	PHP 1, 279.29
	Total	PHP 1, 279.29	PHP 1, 279.29

Prepared by:

Glenn Marco B. Accad
Adrian V. Gallano
Jefferson G. Mamauag
Jose Mari Christian V. Tayag

Noted by:

Genedina B. Accad **Annabelle V. Gallano**
Mary Grace G. Mamamuag **Rogelio G. Tayag**

Approved by:

Norvin Shaq F. Lintag **Enoch James C. Tayaguna**

APPENDIX C. USER'S MANUAL





SandEE



An Educational Multi-Platform 2D Open World Survival Crafting Game for Lua and Python for ICT and STEM

ABSTRACT

SandEE is a two-dimensional sandbox game about gathering, building, survival, exploration and learning to program. This manual is the official player guide for the Windows and Android editions. It explains how to start a world, learn the controls, gather and craft, survive the overworld, clear the dungeon missions and complete the mission tracks.

The manual follows the game from first launch to the later missions. Chapter 2 describes how to play: the mission tracks, how stars are earned and a practical routine for clearing dungeons. Later chapters cover controls, inventory, crafting, caves and troubleshooting. The two programming chapters list every custom Automatica and Algorithmica function so players can automate work once they know the game.

Screenshots throughout the manual show the interfaces and actions described. Function tables give the exact name, arguments and effect of each custom function, and the contents and cross-references let players jump straight to the system they need.

CHAPTER 1. INTRODUCTION

1.1 About SandEE

SandEE is a two-dimensional sandbox game that combines gathering, building, survival, exploration and introductory programming. Players explore a generated world, gather resources, craft tools, build structures, farm, fish, fight creatures and descend into caves and dungeons. The world is designed to support both short play sessions and long-term building projects.

Two programming environments are part of the game. Automatica gives players a programmable Lua robot that performs work in the world. Algorithmica gives players a Python-based interface for automating crafting stations and furnaces. Dungeons introduce programming concepts through short lessons, Knowledge Altars, quiz questions and practical code exercises.

SandEE is available for Windows and Android. The two editions share the same world rules, crafting systems and programming environments, but they use different control schemes. This manual identifies platform differences wherever they affect how a feature is used.

1.2 Purpose and Audience

This manual is the official reference for players of SandEE. It explains what each feature does, how to reach it, what the player should expect after using it and which restrictions apply. It is written for new players who need a guided introduction, for experienced players who need a quick control or programming reference, and for instructors who use SandEE as a classroom or workshop tool.

The manual assumes no previous experience with SandEE. It does assume that the reader can use a keyboard and mouse or a touch screen, and that the reader can follow short code examples. Programming chapters explain the game-specific functions before presenting complete programs.

1.3 Using This Manual

Screenshots are included beside the instructions they illustrate. A figure may show a menu, a world interaction, a programming editor or the result of an action. When an action changes the interface, the manual shows both the initial state and the result where a screenshot is available.

Code examples use a monospaced block. Function names, parameters and interface labels are also identified in the surrounding text. The examples are complete enough to run, but players should read the accompanying explanation before changing recipe names, item names or quantities.

Platform differences are marked in the text and in comparison tables. Unless a section states otherwise, a feature is available on both Windows and Android. Android instructions use touch terminology such as tap, hold and drag; Windows instructions use click, hold and drag.

1.4 Supported Platforms and Known Limitations

SandEE supports Windows desktop and Android touch devices. The Windows edition uses keyboard and mouse controls. The Android edition uses an on-screen joystick, action buttons and touch gestures. Both editions support the same world types, crafting systems, dungeons and programming environments.

Some account features depend on the online service. SandEE uses username-and-password authentication. Email registration, Google sign-in and password recovery are not available. Cloud saves, leaderboards and account switching require an active online session. A previously verified offline world can be played without a live session, but new worlds must be created online.

Algorithmica accepts a restricted subset of Python syntax. Imports, f-strings, list comprehensions, function definitions, comparison operators and else branches. The programming chapter explains the supported syntax and the recommended way to write station and furnace programs. The Android screenshots in this manual show the mobile interface in the game's mobile mode. Physical-device cutouts, gesture navigation and on-screen keyboard placement can vary by device.

1.5 Chapter Overview

Chapter 2 explains how to play: the mission tracks, the star system and a practical dungeon routine. Chapter 3 covers installation, startup and the account. Chapter 4 explains sign-in and account management. Chapter 5 covers the main menu and world management. Chapter 6 describes the gameplay interface. Chapters 7 and 8 document Windows and Android controls. Chapter 9 covers exploration, gathering, combat and waypoints. Chapter 10 covers inventory and items. Chapter 11 covers crafting and stations. Chapter 12 covers dungeons and missions. Chapter 13 covers caves. Chapters 14 and 15 list the Automatica and Algorithmica functions. Chapter 16 covers settings and help. Chapter 17 covers troubleshooting. Chapter 18 concludes the manual, and Chapter 19 contains the appendices.

CHAPTER 2. HOW TO PLAY SANDEE

2.1 What You Can Do

SandEE is a sandbox with no timer and no game-over screen. Play at your own pace: explore the world, gather resources, build a base, craft equipment, farm, fish, fight creatures and enter dungeons. The dungeon missions are grouped into the Python track and the Lua track, and twenty side missions add optional goals.

Progress is recorded by the mission tracker and the star summary. The tracker lists the missions you can take next, and the stars record how cleanly each dungeon was cleared. A zero-star clear still counts, so a difficult run never blocks the next mission.

2.2 Mission Tracks

Table 2.1. Dungeon mission tracks.

Track	Content	Final state
Python	Print, integers, variables, strings, conditionals, loops, functions and related concepts.	Unlocks later Python missions as prerequisites are met.
Lua	Lua print, variables, tables, loops, functions, conditions and automation concepts.	Lists every Lua dungeon.
Side missions	World, cave, combat, fishing and automation objectives.	Optional goals; remain available alongside the tracks.

2.3 Earning Stars

Every dungeon starts with three blue stars. A hint or a wrong answer removes one, and the first main dungeon offers free hints. The run still clears at zero stars and still grants its reward. Yellow stars are mission rewards, and a three-star clear adds a perfect-clear bonus. Figure 12.3 shows a two-star result.

Table 2.2. Dungeon quiz scoring.

Event	Star effect	Notes
Start a dungeon	3 blue stars available	Stars are awarded for the run, not stored as currency during the run.
Reveal a hint	−1 star	Free in the first main dungeon.
Wrong compiler output	−1 star	The question does not advance.
Wrong multiple-choice answer	−1 star	The question does not advance.
Clear all questions at 0 stars	Clear still counts	The reward and mission completion are granted.
Clear all questions at 3 stars	Perfect clear	Recorded as the dungeon's best result.

Blue stars measure how cleanly a dungeon was solved; yellow stars measure how much of the mission list is complete. Both totals appear in the top-right of the HUD.

2.4 A Play Routine

- Finish the tutorial or skip it, then craft a basic axe, pickaxe and sword.
- Place a Bonfire near your base and keep it active. It is your respawn point.
- Activate each Skyglass Waypoint you pass. Fast travel shortens every supply trip.
- Carry food in the hotbar. Select it and click yourself to eat and restore health; on Android, tap yourself.
- Enter caves and dungeons with a full set of tools. The knowledge room in a dungeon heals you and keeps enemies out.
- Answer quiz questions before spending a hint; hints are free only in the first dungeon.
- After both tracks, use Automatica and Algorithmica to automate wood, ore and crafting while you finish the side missions.

CHAPTER 3. INSTALLATION AND STARTUP

3.1 Obtaining the Game

SandEE is distributed as a compressed Windows archive and an Android APK. Extract the Windows archive and run the executable from its folder. Install the Android package through the device's normal package installer.

3.2 First Launch and the Startup Gate

The game opens a startup gate before the title menu. The gate checks whether the device is online and whether SandEE's online service responds. The gate is shown in Figure 3.1 when both checks succeed. The primary button reads Sign In / Play Online. The status line reports Online services available, and a second line explains that signing in synchronises worlds, cloud saves and leaderboards.



Figure 3.1. Startup gate with online services available.

When either check fails, the gate disables the online button and displays a retry control. The status line reads Internet connection required. The panel still notes that a previously verified offline copy remains available, but the startup gate itself does not provide an offline-play button; offline entry is offered later from the main menu's online controls.

The startup gate has three interactive elements: the Sign In / Play Online button, the Retry button that appears when the check fails, and the account form that opens after a

successful check. Pressing Retry re-runs the connectivity check and re-enables the online button if the service becomes reachable. Pressing the online button opens the shared account panel described in Chapter 4. If the panel is cancelled, the gate remains available for another attempt.

3.3 Loading a World

After a world is selected, the game shows a dedicated loading screen while terrain, foliage, structures, waypoints, mobs and the player are prepared. The screen displays the title SandEE Alpha, a status line such as Growing grass… 7% and a progress bar. The loading screen is removed with a short fade after generation completes. The player remains frozen until the fade ends, so input cannot act through the overlay.

SandEE generates a large overworld. Loading time depends on the device and the amount of content being created. The loading screen reports progress by phase, so the percentage can pause briefly while a heavy phase runs.

3.4 Startup Failure Behaviour

If the online service is unreachable, the gate reports the failure and offers Retry. If a saved session can no longer be validated, the game returns to the sign-in form. A network outage does not sign the player out. If the player cancels the account panel, no world is loaded and the gate remains in its current connectivity state.

CHAPTER 4. ACCOUNT REGISTRATION AND AUTHENTICATION

4.1 Signing In

SandEE uses a username and password. Sign-in, registration and the signed-in account view share one account panel.

Figure 4.1 shows the sign-in form. Enter a username and password and press Sign In; an empty field produces Enter your username and password. A failed attempt reports Incorrect username or password. or a connection message.



Figure 4.1. Sign-in form in the shared account panel.

4.2 Creating an Account

Figure 4.2 shows the registration form. Choose a username of 3–24 letters, numbers or underscores, and a password of 6–72 characters, then repeat the password. The Show Password toggle reveals both fields.

The rules are checked before the username is sent, so an invalid form is rejected with an inline message. If the username is already taken, the form asks for another one.



Figure 4.2. Registration form with username, password and confirmation fields.

4.3 Signing Out and Playing Offline

The account control in the main menu opens the signed-in profile and provides Sign Out. Sign out before switching accounts.

A previously verified world can be opened without a live connection. New worlds, cloud saves, leaderboards and account switching require an online session.

CHAPTER 5. MAIN MENU AND WORLD MANAGEMENT

5.1 Main Menu Layout

The main menu presents a large SandEE logo. The footer reports the current mode, for example Offline Local Copy or Offline. The background is a slowly panning deterministic world panorama.

Figure 5.1 shows the menu in verified offline mode. The buttons are Continue, Worlds, Go Online, Settings and Quit. With an online session, the online control becomes the account/online entry point and the status area reports the online state.



Figure 5.1. Main menu in verified offline mode with Continue enabled.

The main menu actions are summarised in Table 5.1. The Continue button is enabled only when a verified account is active and at least one readable world exists; it loads the most recently saved world. The Worlds button opens the world manager. The online button opens the shared account panel when offline, or the account/online controls when online. Settings opens the shared settings panel. Quit closes the application immediately on the main menu; no confirmation dialog is shown there.

Table 5.1. Main menu actions.

Control	Action	Result	Prerequisite
Continue	Load the most recent world	Loads the most recent save and opens the world.	A verified account and a readable world.
Worlds	Open the world manager	Shows world cards, rename/delete controls and the create form.	A verified account.
Go Online	Open the account/online flow	Checks connectivity, restores a cached session or shows the sign-in form.	None; failure leaves offline play available.

Control	Action	Result	Prerequisite
Play Offline	Switch an online session to a verified offline copy	Re-reads local worlds and updates the status label.	An online session and a verified offline copy.
Settings	Open Settings	Shows Audio, Display, Gameplay and Controls categories.	None.
Quit	Close the game	Exits the application.	None.

5.2 World Selection

The Worlds overlay is shown in Figure 5.2. Each world card displays the world name and a last-saved timestamp. A pencil button opens the rename dialog, and a red Delete button opens the delete confirmation. The lower portion of the overlay contains a Create a new world section with a name field and a Create button. The Back button closes the overlay.



Figure 5.2. World manager with world cards and the create-world form.

World creation is online-only. In offline mode the create field is visible but the form reports Offline play cannot create new worlds. Reconnect to add a world. The Create button remains disabled. When the account is online and cloud-verified, a unique name enables the button. Creating a world uses the lowest free slot, opens the world and generates a new world. A world name must be non-empty, unique across the account's worlds and no longer than 40 characters in the rename dialog; the create field itself is validated by the same name-free check.

A duplicate name produces the inline message “That name is already used by another world.” and keeps the Create button disabled. An empty name produces Enter a name for your world. and the same disabled state. The game supports at most ten worlds per account; once the limit is reached, the next-free-slot lookup returns no slot and creation is refused.

5.3 Rename and Delete

The rename dialog is opened from the pencil button on a world card. It contains a name field, a Cancel button and a Rename button. The same free-name rule applies. A successful rename updates the world card and the save metadata without touching any other slot.

Deletion always asks for confirmation. The message includes the world name and the warning This cannot be undone. Cancel is the default action. Confirming deletes the world and records the deletion when online.

5.4 Main Menu Failure Behaviour

If no world exists, Continue is disabled; open Worlds to create one. If the account is offline, the menu keeps the last verified world playable and reports the status in the footer. If the online check fails, Retry runs it again. Quit closes the game normally.

CHAPTER 6. GAMEPLAY INTERFACE

6.1 The Gameplay HUD

The gameplay HUD is shown in Figure 6.1. It remains visible during normal exploration and is dimmed or hidden when a modal panel opens. The HUD is composed of the following elements: the minimap in the upper-left, the Automatica status panel in the left-middle, the pause button and star summary in the upper-right, the mission tracker below them, the health and stamina bars above the hotbar, the nine-slot hotbar at the bottom-centre, the selected-item label above the bars and the contextual control hints in the lower-right.



Figure 6.1. Gameplay HUD during overworld exploration.

Table 6.1 describes each element and its behaviour. The layout adapts to different window sizes and to Android safe areas. On Android, the hotbar reserves its last slot for the INV button, leaving eight item slots; the desktop hotbar has nine item slots.

Table 6.1. Gameplay HUD elements.

Element	Location	Purpose	Interaction
Minimap	Upper-left	Shows explored terrain, the player, creatures, structures and waypoints.	Click or press M to open the full map.
Automatica status	Left-middle	Lists the most recent Automatica units and their current job.	Click the panel to expand the full list where available.
Star summary	Upper-right, left of Pause	Shows current yellow and blue star totals.	Read-only.
Pause	Upper-right	Opens the pause menu.	Click or tap.
Mission tracker	Right side	Shows active Lua, Python and side missions with progress.	View All Missions opens the full list.

Element	Location	Purpose	Interaction
Health bar	Bottom-centre	Shows current and maximum health.	Read-only.
Stamina bar	Bottom-centre	Shows current and maximum stamina.	Read-only; sprinting drains it.
Selected item label	Above health bar	Names the currently selected hotbar item after a change.	Read-only; fades after a short time.
Hotbar	Bottom-centre	Holds usable items and blocks.	Number keys, mouse wheel or click/tap.
Control hints	Lower-right	Shows movement, sprint, LMB and build bindings; keycaps light while pressed.	F1 toggles on desktop.
Notifications	Top-centre	Reports pickups, fishing, deaths, discoveries and warnings.	Read-only; fades.
Autosave indicator	Top-centre	Reports Autosaving… saving progress.	Read-only.

6.2 Onboarding and the Tutorial Guide

A new world begins in a guided camp. The tutorial director introduces the game through Py-San, a friendly guide, and then presents one short objective at a time. Figure 6.2 shows the first dialogue. The dialogue box displays the speaker name, the text, a portrait, a SKIP LESSON button and the instruction Click or Enter to continue, Esc closes. The world is modal while the dialogue is open.



Figure 6.2. Py-San's opening dialogue in a new world.

After the dialogue is advanced, the objective card appears at the top of the screen and the target is highlighted in the world. The movement lesson card reads GUIDED

START, Use WASD or the arrow keys to walk to the marker., a large arrow points toward the target and the destination is outlined with a pulsing marker. The player can skip the lesson with the SKIP LESSON button; skipping does not remove items, features or later content.

The onboarding sequence covers movement, tool selection, tree chopping, wood pickup, crafting-station placement, the first dungeon, the Knowledge Altar, the Quiz Altar, Automatica, Algorithmica, regular caves, Pixie Caves and Skyglass Waypoints. The opening HUD is deliberately reduced and expands as lessons complete. After onboarding, a settings tour highlights the main controls. The tutorial state is stored per world, so an existing world is not forced through onboarding again.

6.3 The Full Map

The minimap expands into the full map shown in Figure 6.3. The map header reads MAP, with the instructions drag to pan • CLOSE / M to close and a CLOSE button. The map canvas shows explored terrain in colour and unexplored areas in dark grey. The legend identifies the player, enemies, friendly creatures, bonfires, Automatica units, crafting stations, furnaces, Color Crafters, dungeons and world waypoints.



Figure 6.3. Full map with legend and explored terrain.

The map supports dragging to pan and mouse-wheel or plus/minus zoom. The M key toggles it. Markers can be clicked or tapped to travel when the marker type supports travel; the world-waypoint map described in Chapter 9 is the dedicated travel surface. Closing the map returns to gameplay without changing the world state.

6.4 Pause Menu

The pause menu contains Resume, Help & Documentation, Settings, Automatica Management and Quit to Menu. Automatica Management lists every active Lua unit and is the only player-facing way to terminate one. Quit to Menu saves the current world through the normal cleanup path, leaves any cave or dungeon safely and returns to the title menu.

Escape opens the pause menu when no other panel is open and closes the innermost panel when one is open. On Android, the system Back action follows the same overlay order. The pause menu is modal; gameplay input is ignored until it is closed.

6.5 Death and Respawn

When health reaches zero, the game shows a red death banner and starts a five-second respawn countdown. Figure 6.4 shows the state at zero health. The player's inventory is not deleted. Build mode and pending actions are cancelled. The player respawns in the overworld at the active Bonfire, or at the world spawn if no Bonfire is active. A death inside a cave or dungeon returns the player to the overworld; a dungeon death restarts at the dungeon entrance rather than deleting the run.



Figure 6.4. Death overlay and five-second respawn countdown.

6.6 Notifications and Autosave

Short notifications appear at the top-centre of the screen. They report item pickups, bestiary discoveries, fishing casts, farming results, deaths and validation warnings. The autosave indicator appears in the same region and reads Autosaving… saving progress while a save is being written. SandEE saves automatically at regular intervals and after mission progress changes. The indicator does not interrupt play.

CHAPTER 7. WINDOWS CONTROLS

7.1 Windows Control Reference

Table 7.1. Windows keyboard and mouse controls.

Action	Input	Behaviour
Move	W, A, S, D or arrow keys	Moves in the chosen direction; cancels point-and-click.
Sprint	Shift	Runs while stamina remains.
Aim	Mouse movement	Sets the direction of swings, tools and placement.
Primary action	Left click	Swings, mines, chops, attacks, interacts or places.
Point-and-click	Left click on ground or target	Walks to the target and performs the action when enabled.
Inventory	E	Opens or closes the inventory.
Hotbar	1–9 or mouse wheel	Selects a hotbar slot.
Build mode	Ctrl	Toggles the build grid and ghost preview; unavailable in dungeons.
Map	M	Opens or closes the full map.
Pause	Escape	Opens the pause menu or closes the innermost panel.
Eat food	Left click yourself with food selected	Restores health when health is not full.
Destroy object	Shift + left click	Mines a placed block, chest, station or furnace.
Drag and drop	Left mouse drag	Moves stacks between inventory, hotbar and chests.
Dialogue	Enter or Space	Advances tutorial dialogue.

7.2 Movement, Building and Interaction

Point-and-click movement is enabled in Settings → Controls. A click on open ground walks there; a click on an object walks into range and uses it; a click on an enemy chases and attacks. WASD or Escape cancels the order.

Ctrl shows the build grid. Valid tiles appear in the block's colour and blocked tiles in red. Placement is limited to six blocks, and build mode is unavailable in dungeons. Figure 7.1 shows the grid.



Figure 7.1. Build mode with placement grid and invalid red cells.

A plain click on a chest docks its slots beneath the inventory, and items move by dragging between the two grids. Figure 7.2 shows the combined view. Stations, furnaces and Color Crafters open their own menus. Mining, chopping and fishing use click or hold actions, and Escape closes the innermost panel or opens the pause menu.



Figure 7.2. Chest storage docked beneath the player inventory.

CHAPTER 8. ANDROID CONTROLS

8.1 Mobile HUD

The Android HUD replaces the desktop keyboard hints with a virtual joystick and action buttons. Figure 8.1 shows the layout: the joystick is in the lower-left, BUILD, SPRINT and INTERACT are in the lower-right, and the hotbar has eight item slots plus

an INV button. The minimap, star summary, mission tracker, health and stamina bars remain in the same logical positions as on desktop.



Figure 8.1. Android control layout with joystick, action buttons and INV slot.

SandEE displays the mobile layout when it runs on Android. The Android screenshots in this manual show that layout, including the joystick, action buttons and the eight-slot hotbar with INV. Device-specific cutouts, gesture navigation and on-screen keyboard placement can vary by device.

8.2 Touch Controls

The movement joystick appears in joystick mode. A second finger can use the action buttons and hotbar while the first finger holds the joystick, so movement and actions are multitouch. SPRINT is a hold control. BUILD toggles build mode and changes its label to BUILD ON. INTERACT is contextual: it reads OPEN near a chest, CRAFT near a crafting station and SMELT near a furnace. Figure 8.2 shows the button reading OPEN while the player stands beside a chest.



Figure 8.2. Android INTERACT button showing the contextual OPEN label.

Tapping the world performs the same action as a desktop left click: it moves under point-and-click, mines or chops the tapped target, attacks a tapped hostile creature, casts a fishing rod at tapped water or places a block in build mode. Tapping the player while food is selected eats the food. The inventory opens from the INV slot or the INTERACT button where applicable. The inventory keeps the hotbar visible below the panel.

Build mode on Android shows the same translucent ghost and tile grid as desktop and changes the build button label to BUILD ON. The map button on the minimap opens the full map. The system Back action closes the current panel or menu in the same order as Escape on desktop.

CHAPTER 9. OVERWORLD GAMEPLAY

9.1 Exploration and Biomes

The overworld contains plains and forest, desert, snowy land, water, caves and structures. The player explores on foot, by point-and-click or by joystick. A full day/night cycle lasts 24 real minutes: 12 minutes of day and 12 minutes of night. Night increases the hostile population cap and spawn rate. Some creatures appear only at night. Desert entry spawns a group of Desert Worms, and snow biomes can trigger a snowstorm that reduces visibility and movement speed to 55 percent until the player leaves the snow or night falls.

The minimap records explored tiles as the player moves. The full map and the Skyglass Waypoint map are described later in this chapter. The world remains open after the tracks are complete; building, farming, fishing, automation and side missions remain available.

9.2 Trees and Wood

Trees are the first resource interaction in the tutorial. A tree can be chopped without an axe, but an axe shortens the break duration; each tree type drops its own wood. Figure 9.1 shows the player aiming at a tree with the iron axe selected. When the tree falls, it drops wood items on the ground. Walking over a dropped item collects it if there is inventory space; otherwise it remains on the ground.



Figure 9.1. Player preparing to chop a tree with an axe.

9.3 Mining and Ores

Placed blocks and cave walls use a hold-to-break model with a minimum pickaxe tier. The wooden pickaxe can mine stone, iron and copper, the iron pickaxe can mine diamonds, the diamond pickaxe can mine everything and is the fastest.

Figure 9.2 shows a placed iron-ore block. Holding the correct pickaxe breaks the block, and the dropped ore is collected by walking over it.



Figure 9.2. Placed iron-ore block before mining.

Mining is tool-gated by block tier. If the held tool is below the block's minimum tier, the block does not break and the game reports that a better pickaxe is required. A placed block can also be destroyed with Shift+click on desktop, subject to the same tier rule. Release the button to cancel; hold it to chain-break the next block under the cursor after the current one breaks.

9.4 Farming

Farming uses a hoe, wheat seeds and tilled ground. A hoe click on open, dry ground within 20 blocks of water creates a tilled plot. A seed click on the plot plants wheat. Wheat matures after 180 seconds; a sword cut harvests a mature crop and may also return a seed. Figure 9.3 shows a mature wheat crop on tilled ground with the iron hoe selected and the Tilled Ground pickup notification visible.

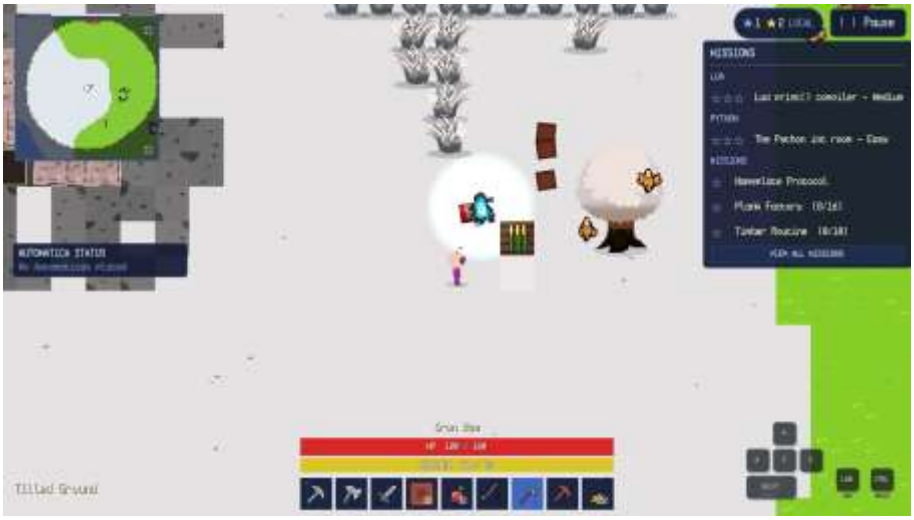


Figure 9.3. Mature wheat on a tilled plot.

Tilling fails if the tile has no ground, is water, is occupied by a placed block, tree, cave entrance or dungeon entrance, or is more than 20 blocks from water. Planting fails if the target is not tilled ground or already contains a crop. A sprout cannot be harvested; the sword cut leaves it growing.

9.5 Fishing

Fishing requires a fishing rod in the selected hotbar slot and a click or tap on water within eight blocks. The controller draws a slack line and a bobbing float, as shown in Figure 9.4. The cast lasts five seconds and then rolls a catch chance between 70 and 90 percent. A successful catch reels a fish to the player along an upward arc; a failed attempt ends without an item. Moving more than 64 pixels from the cast point, switching items or leaving the area cancels the attempt.



Figure 9.4. Fishing line and float after a successful cast.

Casting on a non-water tile swings the rod instead. Casting beyond the range produces Too far to cast the rod reaches 8 blocks. and does not start the timer. A second cast while a line is active is ignored.

9.6 Waypoints and Fast Travel

Skyglass Waypoint towers are generated across the world. Towers are spread across the world; activate each one you find. A tower displays its name and the instruction click to open waypoint map. Interacting with the tower opens the world-waypoint map. The map header reads WORLD WAYPOINTS; the legend distinguishes the player, the current waypoint and other world waypoints. A tower can be used again after the map closes. Waypoints are persistent per world. Their identities and positions are saved with the world, so a reload restores the same network. Dungeons also contain temporary one-way waypoints near the player; these target permanent world towers and are described with dungeon navigation.

9.7 Bonfires and Respawn

A Bonfire is a placeable respawn anchor. Placing and activating one sets the player's respawn point; only one Bonfire is active at a time. Breaking the active Bonfire clears the respawn point and returns the player to the world spawn on the next death. The Bonfire is also a light source and uses the same placement and destruction rules as other placeable objects.

9.8 Creatures and Combat

The world is home to creatures such as Wolf, Sheep, Desert Worm, Headless Fish, Pixie and several dungeon enemies. Hostile creatures approach and attack the player; passive creatures flee or wander. A creature that takes damage shows a name and health bar for a short time, and a damage number appears at the hit location. Figure 9.5 shows a wolf after a hit, with the player's sword selected and the wolf's health state visible.



Figure 9.5. Creature combat

Sword can cut grass.

9.9 Other Overworld Interactions

Table 9.1 summarises the remaining overworld interactions.

Table 9.1. Overworld interactions.

Object or action	How to interact	Result	Failure behaviour
Grass tuft	Walk through or swing a sword	Walkable decoration; sword cut may drop wheat seeds.	No collision; no failure.
Tree	Hold left click / tap with any tool; axe is faster	Drops wood; completes tree mission.	Out of range or wrong aim cancels.
Placed block or ore	Hold left click with a sufficient pickaxe	Breaks and drops the block.	Insufficient tier reports a tool requirement.
Dropped item	Walk over it	Collected into inventory or hotbar.	Full inventory leaves it on the ground.
Tilled plot	Hoe click on suitable ground	Creates farmland.	Too far from water, occupied or invalid ground fails.
Wheat crop	Sword cut when mature	Drops wheat and possibly seeds.	Sprout is left growing.

Object or action	How to interact	Result	Failure behaviour
Water	Fishing rod click within range	Starts a five-second cast.	Non-water or out-of-range cast fails.
Bonfire	Place and activate	Sets respawn point; provides light.	Only one active; breaking clears it.
Chest	Left click	Docks storage beneath inventory.	Shift+click destroys instead.
Crafting station	Left click	Opens crafting and Algorithmica.	Out of range or UI open blocks.
Furnace	Left click	Opens smelting and Algorithmica.	Out of range or UI open blocks.
Color Crafter	Left click	Opens colour mixing.	Out of range or UI open blocks.
Lua station	Left click	Opens Automatica editor.	Combat mode blocks code execution.
Waypoint tower	Left click	Opens world-waypoint map.	Out of range or UI open blocks.
Dungeon portal	Left click	Opens enter/cancel confirmation.	Cancel leaves the world unchanged.
Cave entrance	Left click	Opens enter/cancel confirmation.	Cancel leaves the world unchanged.
Creature	Left click with a weapon	Attacks; hostile creatures retaliate.	Out of range or no weapon still swings.

CHAPTER 10. INVENTORY AND ITEM MANAGEMENT

10.1 Inventory Layout

The inventory panel is shown in Figure 10.1. It has four tabs INVENTORY, CRAFT, ENCYCLOPEDIA and BESTIARY a CLOSE button, a grid of twenty slots, a details sidebar, a trash drop area and the persistent hotbar below the panel. The hotbar has nine slots on desktop and eight item slots plus INV on Android. The inventory itself has twenty slots; each slot can hold a stack whose limit is defined by the item, with a global default of 32 and a limit of one for tools.



Figure 10.1. Inventory grid, details sidebar and trash area.

Selecting a slot shows the item's name, quantity, type, effect and description in the sidebar. For a Wood stack, the panel shows Qty: 16 / 32, type Block, and the description Basic crafting material. A plain click selects and inspects; it does not move the stack.

10.2 Drag and Drop

Dragging a stack from one slot to another swaps the two stacks. The source slot is marked as the drag source, the target slot is outlined in yellow and a semi-transparent preview follows the pointer. A semi-transparent preview follows the pointer while the source stack is still visible. Dropping onto an empty slot moves the stack; dropping onto a compatible stack merges up to the stack limit; dropping onto an incompatible stack swaps. Dropping onto the trash icon deletes the dragged stack. The chest view uses the same slot component, so items can be dragged between the player inventory

and chest storage in either direction.

The inventory does not provide right-click splitting. Stack splitting is performed by dragging a stack onto an empty slot and then rearranging; there is no dedicated split control. This is a limitation of the current interface, not an omitted instruction.

10.3 Craft Tab

The CRAFT tab lists recipes that can be crafted without a crafting station. The four category buttons are Materials, Blocks, Weapons & Tools and Others. Selecting a recipe shows its ingredients, the result and a craft button. The button reads CRAFT when the player has the required items and MISSING MATERIALS otherwise. Figure 10.2 shows the Blocks category with the Color Crafter recipe selected; the cactus requirement is red because the player has 16 of the required 20.



Figure 10.2. Inventory craft tab with a recipe and missing-material state.

Station-required recipes do not appear in the inventory craft tab; they are listed in the crafting-station menu. The inventory recipe set is therefore smaller than the full recipe set.

10.4 Encyclopedia

The ENCYCLOPEDIA tab is the Programming Encyclopedia. It contains four accordion sections: LUA Programming Language, Automatica LUA Functions, Python Programming Language and Algorithmica Python Functions. Language entries unlock through dungeon knowledge; the two function sections are always available. Figure 10.3 shows the Automatica section expanded with the full function list.



Figure 10.3. Programming Encyclopedia with Automatica functions expanded.

The counts in the section header show the number of entries currently discoverable and the total entry count. The in-game encyclopedia lists 34 Automatica entries, which corresponds to 30 functions plus four read-only variables. The language sections show 0 / 15 for Lua before dungeon discovery; the count increases as knowledge entries are learned.

10.5 Bestiary

The BESTIARY tab records creature sightings and defeats. A sighting reveals the creature's name, habitat and silhouette. A defeat reveals its field notes, maximum health and complete drop table. The header reports 1 sighted • 0 studied, and the detail panel explicitly says Defeat this creature once to reveal its field notes, maximum health and complete drop table. The Bestiary is per world, so a new world starts with an empty record.

10.6 Item and Stack Rules

Every item has a name, type, icon, stack size and, where relevant, combat, food or smelting data. Tools do not stack; most materials stack to 32. A full inventory causes new pickups to remain on the ground or, for mission and dungeon rewards, to enter the pending-overflow queue and be dropped beside the player when space becomes available. The game never silently deletes a reward when the inventory is full.

Table 10.1. Inventory slots and stack rules.

Container	Slots	Notes
Hotbar (desktop)	9	Tools, blocks, food and utility items.
Hotbar (Android)	8 item slots + INV	The last slot is reserved for the inventory button.
Player inventory	20	Grid with a details sidebar and trash area.
Chest storage	8	Docked beneath the player inventory while open.
Stack limit	Item-defined, default 32	Tools have a limit of one.
Overflow	Pending queue	Rewards are dropped beside the player when space becomes available.

CHAPTER 11. CRAFTING AND STATIONS

11.1 Crafting Overview

Crafting is split between hand crafting in the inventory and station crafting at a Crafting Station. A recipe has a result item, a result quantity, up to two ingredients, a category and a station requirement. The current recipe catalogue contains 48 recipes. Hand crafting excludes station-required recipes; station crafting includes them. Both interfaces show the same ingredient, result and availability rules.

11.2 Inventory Crafting

The inventory craft tab is described in Chapter 10. Figure 10.2 shows a recipe with a missing ingredient and a disabled action. When all ingredients are present, the action button becomes CRAFT and consumes the ingredients from the hotbar and inventory. The new item is added to the inventory or dropped beside the player if the inventory is full. Crafting completes the craft_item mission objective and contributes to side-mission progress.

11.3 Crafting Station

A placed Crafting Station opens the menu shown in Figure 11.1. The left side lists the player's inventory and build slots. The right side lists station recipes by category. Selecting a recipe shows its ingredients and result; the action button reports CRAFT or NOT ENOUGH MATERIALS. The ALGORITHMICA button opens the Python editor described in Chapter 15. The CLOSE button closes the menu.



Figure 11.1. Crafting Station menu with recipe details.

Station recipes include planks, sticks, charcoal, iron rods, tools, furniture and automation items. A recipe can draw ingredients from the station's input slot and from the player's inventory. The station owns its input, output and automation state; the UI only calls the station's public methods. A Lua robot can also operate a nearby station through set_recipe, craft, put_input, input_slot and take_output.

11.4 Furnace

A placed Furnace opens the smelting menu shown in Figure 11.2. The menu has an inventory grid, an input slot, a fuel slot, an output slot and a SMELT button. The input slot accepts smeltable items such as iron ore; the fuel slot accepts wood or charcoal. The status line reports Smeltable when the input and fuel are valid. The ALGORITHMICA button opens the furnace Python editor. The CLOSE button closes the menu.



Figure 11.2. Furnace menu with iron ore and charcoal loaded.

Smelting consumes one input and one fuel per item. The furnace continues to cook while its menu is open. Removing the input or fuel stops the cook. The output slot must be emptied before a new smelt can start when the output stack is full. The furnace can also be loaded by an Algorithmica program; the menu uses drag-and-drop.

11.5 Color Crafter

The Color Crafter is a placeable station that creates solid-colour blocks from a cactus-based economy. The colour wheel selects hue and saturation; the vertical Value slider controls brightness. The R, G and B fields accept 0–255 integers, and the preview swatch shows the resulting colour. The Craft button reads Craft (uses 1 Cactus) and is enabled only when the player has at least one cactus. Each crafted block consumes one cactus. The Algorithmica craft_color(r, g, b, amount) function implements the same economy from code.

Pure black and pure white are rejected; the menu reports Pure black and pure white are not allowed. The output goes to the station's output slot in code mode; the manual UI adds the block directly to the player inventory, or drops it if the inventory is full.

11.6 Station and Recipe Reference

Table 11.1 summarises the crafting-related stations.

Table 11.1. Crafting stations and their interfaces.

Station	Interface	Inputs	Outputs	Programming access
Crafting Station	Inventory grid, recipe list, output section	Player inventory and station input slot	Crafted items	Algorithmica and Lua station helpers
Furnace	Inventory grid, input/fuel/output slots	Smeltable item and fuel	Smelted item	Algorithmica
Color Crafter	Colour wheel, RGB fields, preview	One cactus per block	Solid-colour block	Algorithmica craft_color
Chest	Docked storage grid	Player inventory	Stored stacks	Lua deposit_chest, chest_contents, withdraw

11.7 Crafting Failure Behaviour

A recipe with insufficient ingredients shows MISSING MATERIALS or NOT ENOUGH MATERIALS and does not consume anything. A station recipe that requires a station cannot be crafted from the inventory tab. A furnace with no fuel reports that fuel is required; a furnace with an invalid input reports that the item is not smeltable. A full output slot blocks a new smelt. A full player inventory causes the crafted result to be dropped beside the station or player rather than deleted.

CHAPTER 12. DUNGEONS AND MISSIONS

12.1 Mission Structure

Missions are grouped into the Python track, the Lua track and the side missions. Table 2.1 in Chapter 2 lists the indices and content of each track. Each dungeon links a Knowledge Entry, a set of quiz questions and a reward.

12.2 Finding and Entering a Dungeon

Dungeon entrances are stone gates placed in the overworld. Figure 12.1 shows a Python dungeon entrance. The gate has a glowing portal, a stone frame and a yellow outline when the player is close. A left click opens the confirmation dialog. The dialog names the dungeon and difficulty, asks Enter the dungeon? and provides Enter and Cancel. Cancel closes the dialog without changing the world.



Figure 12.1. Python dungeon entrance in the overworld.

Entering builds the dungeon randomly depending on the seed.

12.3 Dungeon Layout and Navigation

A generated dungeon contains a guaranteed knowledge room, a guaranteed quiz room, and optional chest rooms. Rooms are connected by corridors. The knowledge room is a safe zone: stepping on its green marker makes the player invulnerable, heals one health per second and disintegrates hostiles inside the room. The quiz room is not a safe zone. Traps and hostile guards appear near the quiz altar, and rare chests appear in

loot rooms.

The dungeon waypoint system places a temporary one-way waypoint near the player at the entrance.

12.4 Knowledge Altar

The Knowledge Altar is the teaching station in the first room. The altar has floating pages and a diamond above it. Interacting with it records the mission's Knowledge Entry and unlocks its keyword in the Programming Encyclopedia. The entry for Python print() has a scrollable body and a Continue button. Closing the panel releases the UI pause and leaves the dungeon running.

The Knowledge Altar also provides an archive of every knowledge entry unlocked in the world. The archive is pageable and lists entries in a stable order. A new world starts with no archive entries; the first dungeon adds the first entry.

12.5 Quiz Altar

The Quiz Altar is the orb in the final room. Interacting with it opens the quiz panel. The panel header reports the question number and total, a HINT button, a RUN button for compiler questions and a CLOSE button. The question text is above a code editor and an output console. Figure 12.2 shows the first question of the Python print room: Print "Hello world, this is my first Python code."



Figure 12.2. Compiler question with code editor and output console.

The quiz supports two question types. A multiple-choice question presents up to four answer buttons and grades the selected index. A compiler question presents a code editor, an output console and a RUN button; the program's printed output is compared with the expected output after whitespace and case normalisation.

12.6 Hints and Penalties

Pressing HINT reveals the answer as faded ghost text in the code editor and changes the button to NEXT. The hint does not submit the answer; the player must type the code and press RUN. In the first main dungeon, hints are free; in later dungeons each hint or wrong run removes one blue star. The results screen states the rule: 3 stars start • each hint or wrong run costs 1 • first-dungeon hints are free • 0 stars still clears.

A wrong compiler answer prints the submitted output but does not advance the question. For example, an incorrect program prints wrong answer in the console. The mismatch is recorded as a star penalty and the player can edit the code and run again.

When the output matches, the panel reports Correct!, advances after a short delay and shows the next question. The editor and output are reset for the new question.

12.7 Results, Stars and Rewards

Answering all questions correctly clears the dungeon. The dungeon is completed and the reward is granted before the results screen opens, so the results screen is presentation-only and cannot duplicate progression. Figure 12.3 shows the results for the Python print room: DUNGEON CLEARED, difficulty Easy, Questions answered 2 / 2, Hints used 1, Star penalties 1, Dungeon reward CPU, two blue stars and the button DONE RETURN TO WORLD. A bonus Life Crystal was also dropped nearby, as reported by the top notification.



Figure 12.3. Dungeon cleared results screen with two blue stars.

Pressing DONE RETURN TO WORLD opens the WHAT YOU'VE LEARNED summary. The summary lists the concepts practised, the key idea (print()), the Python code that was run, the hint usage and the encyclopedia unlock. The CONTINUE button closes the summary, exits the dungeon and returns the player to the overworld entrance.

12.8 Mission Stars and Progression

Completing a dungeon grants the dungeon mission star and a clear star, with a perfect bonus at three blue stars. Yellow stars are mission rewards; blue stars are dungeon-performance rewards. The star summary in the HUD shows both totals. The mission tracker marks a completed dungeon with a check and updates the next available mission. Finishing the Lua track completes the mission list; the twenty side missions remain available alongside and after the tracks.

This manual documents the shared dungeon flow. Later Lua missions use the same altar, quiz and results components; individual walkthroughs for every mission are not included.

CHAPTER 13. CAVE EXPLORATION

13.1 Regular Caves

Regular cave entrances are large rock structures with a dark doorway. Figure 13.1 shows an entrance in a snowy biome. The doorway is outlined in yellow when the player is close. A left click opens the confirmation dialog, which reads Large Cave Entrance and Enter the cave? with Enter and Cancel.



Figure 13.1. Large cave entrance in a snowy biome.

Entering loads a cave with its own walls, ore, lighting, creatures and exit. The minimap switches to cave fog. The exit door returns the player to the overworld entrance. Re-entering a visited cave preserves its state, and saving and loading restores the same cave.

13.2 Pixie Caves

Pixie Caves use a pink stone entrance and a purple interior. The confirmation dialog reads Pixie Cave Entrance and Enter the cave? with Enter and Cancel. The first Pixie Cave is a safe introduction with no pixies; later Pixie Caves are guarded by pixies. Figure 13.2 shows a later cave with ore, crystals and the exit door.



Figure 13.2. Pixie Cave interior with crystals and exit.

13.3 Cave Differences and Persistence

Table 13.1 compares the two cave types. Both use the same entry/exit interaction pattern and the same per-entrance state key, but they differ in art, hazards, creatures and the first-visit safety rule.

Table 13.1. Regular caves and Pixie Caves.

Property	Regular cave	Pixie Cave
Entrance art	Grey rock and dark doorway	Pink stone and dark doorway
Interior palette	Dark blue-grey stone	Purple stone with crystals
Common resources	Stone, iron, copper, cave floors	Pixie stone, crystal ores, pixie floors
Hostiles	Cave creatures and hazards	Pixies; first cave is safe
Exit	Cave exit door	Cave exit door
State	Explored tiles remembered per entrance	Explored tiles remembered per entrance
Save behaviour	Snapshot on exit and autosave	Snapshot on exit and autosave

Cave exploration is saved per entrance. The player's explored-tile set is cleared when a new cave is entered and restored from the cave's snapshot when the player returns. A cave that is left while a panel is open closes the panel through the normal cleanup path; it does not leave the game paused. Death inside a cave returns the player to the overworld spawn or active Bonfire.

CHAPTER 14. AUTOMATICA: LUA ROBOT

14.1 What Automatica Is

Automatica is a programmable Lua robot. Place its station from the Lua item, then click the station or the robot to open the editor. RUN starts the program, STOP halts the current job, CLEAR empties the editor and ENCYCLOPEDIA opens the in-game reference. A robot can chop trees, mine, build, fight, deposit items and operate a nearby crafting station. Standard Lua syntax is available, including variables, tables, loops, functions and print.

Figure 14.1 shows the editor with a program ready to run.

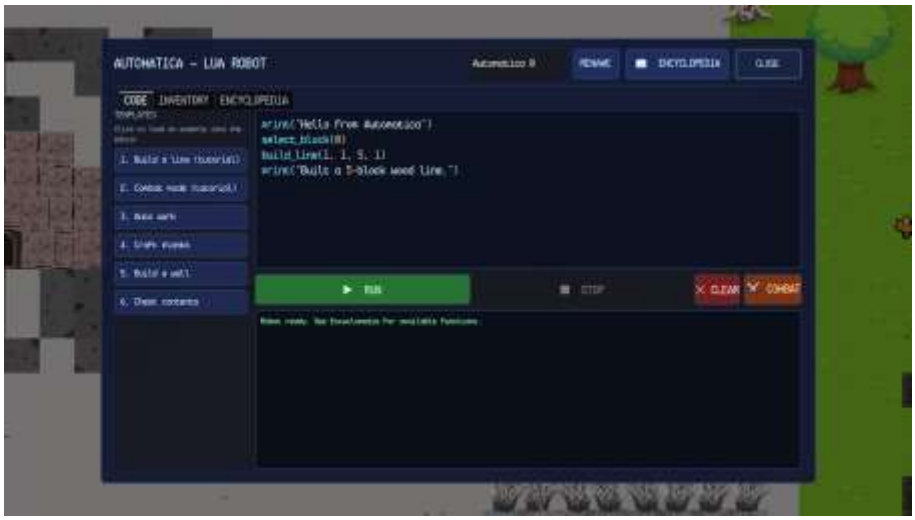


Figure 14.1. Automatica Lua editor with a build program.

14.2 Custom Functions

The tables below list every custom Automatica function. Arguments in brackets are optional; the example column shows a valid call.

Table 14.1. Automatica robot functions (15).

Function and syntax	Parameters and returns	What it does	Working example
auto_work()	No parameters. Returns nothing.	Full work cycle: chop all trees in radius, deposit collected items into the nearest chest, return to the station.	auto_work()

Function and syntax	Parameters and returns	What it does	Working example
chop_trees(radius)	radius is a number of blocks, 1–64; omit for 20. Returns nothing.	Chops every tree within the radius, collects dropped items and walks home. The radius also draws as a blue ring.	chop_trees(12)
deposit_chest()	No parameters. Returns nothing.	Walks to the nearest chest within range, deposits the robot's inventory and returns.	deposit_chest()
chest_contents()	No parameters. Returns nothing; prints the contents.	Prints each item and quantity in the nearest chest, or a message if empty or out of range.	chest_contents()
return_to_spawn()	No parameters. Returns nothing.	Walks directly back to the station and stops.	return_to_spawn()
craft_example()	No parameters. Returns nothing.	Minimal station example: select Planks, craft one and collect it.	craft_example()
move(x, y)	x and y set direction and speed; positive x is east, positive y is south. Returns nothing.	Moves the robot. move(0, 0) stops it.	move(90, 0)
interact()	No parameters. Returns nothing.	Runs the robot's default job: chop all trees and return home.	interact()
attack()	No parameters. Returns nothing.	Swings the robot's held weapon. Requires a tool in the tool slot; otherwise prints No tool equipped.	attack()
drop_item(qty)	qty is the maximum number to drop. Returns nothing.	Drops up to qty copies from the selected build slot, or from the general inventory if the slot is empty.	drop_item(3)
place_item(dx = 0, dy = 0)	Optional tile offsets. Returns nothing.	Places one block from the selected build slot at an offset. With no offsets it places at the robot's tile.	place_item(1, 0)
mine(name)	name is the block's internal name. Returns nothing.	Mines every matching placed block in the work radius, paths around walls and skips unreachable blocks. Requires a tool.	mine("stone")
rename(name)	name is a string. Returns nothing.	Gives the robot a custom name shown above it.	rename("Builder")

Function and syntax	Parameters and returns	What it does	Working example
combat_mode(on)	on is true or false. Returns nothing.	true sends the robot into combat mode; it follows the player and attacks hostiles. false returns it to the station.	combat_mode(true)
list_items()	No parameters. Returns a table of item names.	Returns a sorted list of every item name in the game.	for _, name in ipairs(list_items()) do print(name) end

Table 14.2. Automatica building, chest and geometry helpers (10 unique names).

Function and syntax	Parameters and returns	What it does	Working example
select_block(i)	i is 0, 1 or 2. Returns nothing.	Chooses the build slot used by place_item.	select_block(0)
place_at(x, y)	Absolute tile coordinates. Returns nothing.	Places one block from the selected build slot at an absolute tile.	place_at(12, 8)
withdraw("name", n)	Item name or item id, and a maximum quantity. Returns nothing.	Walks to the nearest chest holding the item and takes up to n; placeable blocks go to the build palette.	withdraw("Wood", 10)
build_line(x0, y0, x1, y1)	Start and end tile offsets. Returns nothing.	Gap-free straight line of blocks.	build_line(1, 1, 5, 1)
build_rect(x0, y0, w, h, filled)	Top-left offset, width, height and filled flag. Returns nothing.	Rectangle of blocks; filled = true builds a slab, otherwise a border.	build_rect(0, 0, 4, 3, false)
build_circle(r, filled)	Radius and filled flag. Returns nothing.	Circle centred on the robot; filled disc or outline.	build_circle(3, false)
build_ring(r)	Radius. Returns nothing.	Hollow ring of radius r.	build_ring(4)
build_spiral(r, turns)	Radius and turns. Returns nothing.	Spiral out to radius r.	build_spiral(4, 2)
build_wave(len, amp)	Length and amplitude in tiles. Returns nothing.	Sine-wave fence.	build_wave(6, 1)
build_stairs(n)	Number of steps. Returns nothing.	Diagonal staircase.	build_stairs(4)

Table 14.3. Automatica station helpers and read-only variables (5 functions + 4 variables).

Name and syntax	Parameters and returns	What it does	Working example
craft(n)	Number of crafts. Returns nothing.	Crafts n of the station's selected automation recipe.	craft(2)

Name and syntax	Parameters and returns	What it does	Working example
set_recipe(name)	Output item name. Returns nothing.	Selects the station's automation recipe.	set_recipe("Planks")
take_output()	No parameters. Returns nothing.	Moves the station's finished output into the robot's stores.	take_output()
put_input("name", n)	Item name and quantity. Returns nothing.	Walks to the nearest station and loads n of the item into its input slot.	put_input("Wood", 10)
input_slot(n)	Quantity. Returns nothing.	Loads n of the selected build-slot item into the station input slot.	input_slot(5)
held_item	Read-only table {name, quantity} or nil.	The item currently held by the robot.	print(held_item.name)
station_recipe	Read-only string.	The station's selected recipe name.	print(station_recipe)
station_output_item	Read-only string.	The station's pending output item name.	print(station_output_item)
station_output_count	Read-only number.	The station's pending output quantity.	print(station_output_count)

CHAPTER 15. ALGORITHMICA: PYTHON FOR STATIONS

15.1 What Algorithmica Is

Algorithmica is the Python interface built into a Crafting Station and a Furnace. Open the machine and press ALGORITHMICA. The editor has a code area, RUN, CLEAR and RECIPES; CLOSE returns to the machine. Programs can select a recipe, craft or smelt items and move the output into your inventory.

Algorithmica accepts simple Python statements: variables, print, loops over range, lists and calls to the functions below. Comparisons, else, def and imports are not supported.

Figure 15.1 shows the station editor.

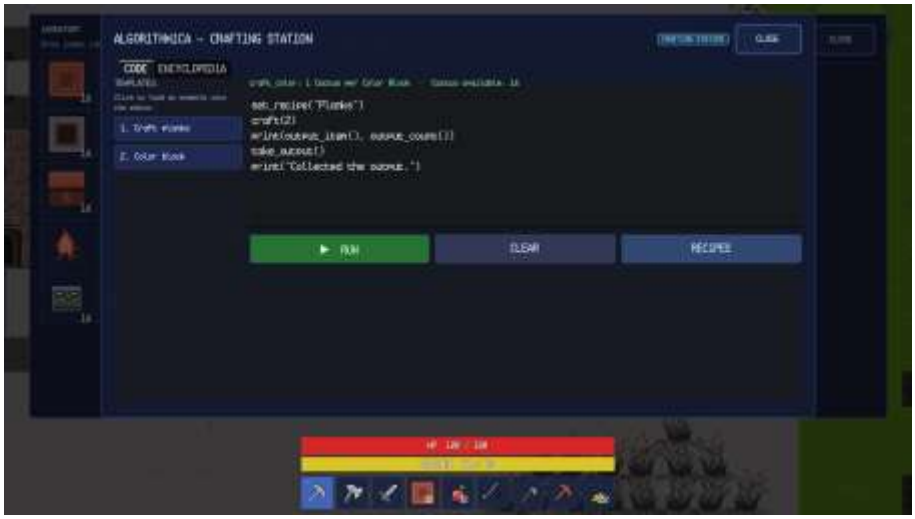


Figure 15.1. Algorithmica Python editor on a Crafting Station.

15.2 Crafting Station Functions

Table 15.1 lists the functions available at a Crafting Station.

Table 15.1. Algorithmica Crafting Station functions.

Function and syntax	Parameters and returns	What it does	Working example
craft(amount)	Integer amount. Returns the number crafted, or 0.	Crafts the selected automation recipe, drawing ingredients from the station input slot first and the player inventory second.	set_recipe("Planks"); craft(2)
craft_color(r, g, b, amount)	RGB integers 0–255 and amount 1–32. Returns the number crafted, or 0.	Crafts solid-colour blocks from cactus, one cactus per block. Pure black and pure white are rejected.	craft_color(255, 0, 0, 2)

Function and syntax	Parameters and returns	What it does	Working example
get_recipe()	No parameters. Returns a string.	Returns the selected recipe name.	print(get_recipe())
set_recipe(name)	Recipe output name. Returns a boolean.	Selects the automation recipe.	set_recipe("Planks")
output_item()	No parameters. Returns a string.	Returns the pending output item name.	print(output_item())
output_count()	No parameters. Returns an integer.	Returns the pending output quantity.	print(output_count())
take_output()	No parameters. Returns the number taken.	Moves the station output into the player's inventory.	take_output()

15.3 Furnace Functions

Table 15.2 lists the functions available at a Furnace.

Table 15.2. Algorithmica Furnace functions.

Function and syntax	Parameters and returns	What it does	Working example
feed(name, n)	Item name and maximum quantity. Returns the number loaded.	Moves up to n of the named smeltable item from the player inventory into the input slot.	feed("iron_ore", 5)
fuel(name, n)	Fuel item name and maximum quantity. Returns the number loaded.	Moves up to n of the named fuel into the fuel slot.	fuel("charcoal", 3)
smelt(n)	Maximum number to smelt. Returns the number smelted.	Smelts up to n, consuming one input and one fuel per item.	smelt(2)
take_output()	No parameters. Returns the number taken.	Moves the output slot into the player inventory.	take_output()
input_name()	No parameters. Returns a string.	Reads the input slot's item name, or "".	print(input_name())
input_count()	No parameters. Returns an integer.	Reads the input slot's quantity.	print(input_count())
fuel_name()	No parameters. Returns a string.	Reads the fuel slot's item name, or "".	print(fuel_name())
fuel_count()	No parameters. Returns an integer.	Reads the fuel slot's quantity.	print(fuel_count())
output_name()	No parameters. Returns a string.	Reads the output slot's item name, or "".	print(output_name())
output_count()	No parameters. Returns an integer.	Reads the output slot's quantity.	print(output_count())

CHAPTER 16. SETTINGS AND DOCUMENTATION

16.1 Settings Overview

Settings is a shared panel used by the main menu and the in-game pause menu. It has four categories Audio, Display, Gameplay and Controls plus Credits and Back buttons. The panel is rebuilt when the UI scale changes, so every category remains usable on a short desktop window and on Android landscape.

16.2 Audio

The Audio category provides three sliders: Master Volume, Music Volume and SFX Volume. Each slider ranges from 0 to 100 percent in 5-percent steps and updates the corresponding audio bus immediately. Master Volume affects music and sound effects; Music Volume affects the music bus; SFX Volume affects mining, footsteps, creatures and other effects.

Audio volume changes apply immediately but are not saved, so they return to 100 percent on the next launch.

16.3 Display

The Display category provides Camera Zoom from 0.5× to 3.0× in 0.1× steps, a Fullscreen toggle and, on desktop, a VSync toggle. Camera zoom changes the player's camera immediately. Fullscreen and VSync are saved for the next launch. On Android, the Fullscreen control is disabled because the platform owns the window mode, and the VSync control is hidden because the compositor owns presentation timing.

Camera zoom returns to its default on the next launch.

16.4 Gameplay

The Gameplay category provides two toggles. Pause on Inventory pauses the game while the inventory is open. Opaque Quest Background draws the quest tracker on a solid panel instead of floating text. Both toggles apply immediately. Pause on Inventory is a global preference; Opaque Quest Background changes the mission tracker's drawing style.

16.5 Controls

The Controls category contains Point-and-Click Movement, Automatic Sprint and a keyboard rebinding list. Point-and-Click Movement enables click/tap movement and interaction. Automatic Sprint applies the sprint multiplier to point-and-click movement while stamina is available. The keyboard list lets the player click a binding row and press a new key; Escape cancels the rebind. Bindings are saved, survive restarts and are independent of keyboard layout.

Point-and-click is available on both desktop and Android. On Android, enabling it hides the virtual joystick; joystick mode keeps the joystick visible. The setting does not intercept build mode, UI panels or right clicks.

Table 16.1. Settings and persistence.

Setting	Category	Values	Persists	Platform notes
Master Volume	Audio	0–100%	No	Applied to the Master bus immediately.
Music Volume	Audio	0–100%	No	Applied to the Music bus immediately.
SFX Volume	Audio	0–100%	No	Applied to the SFX bus immediately.
Camera Zoom	Display	0.5×–3.0×	No	Applied to the player camera immediately.
Fullscreen	Display	On/Off	Yes	Disabled on Android.
Pause on Inventory	Gameplay	On/Off	Session only	Affects whether the inventory pauses gameplay.
Opaque Quest Background	Gameplay	On/Off	Session only	Changes the mission tracker background.
Point-and-Click Movement	Controls	On/Off	Yes	Hides the Android joystick when enabled.
Automatic Sprint	Controls	On/Off	Yes	Applies only to point-and-click movement.
Key bindings	Controls	Physical keys	Yes	Saved with your settings.

16.6 Help and Documentation

Help & Documentation is opened from the pause menu. It contains nine categories:

Start Here, Controls, Survival & World, Building & Farming, Crafting & Machines, Programming & Automation, Dungeons & Bestiary, Maps, Missions & Saving and Settings & Troubleshooting. The panel has a scrollable body and a Back button. The footer reports the instruction scroll to read.

The Start Here category explains the first session, the HUD, the tutorial, the long-term goals and the fact that the game continues after the tracks are complete. The Controls category is the primary in-game control reference.

The other categories document survival, building, farming, crafting, programming, dungeons, maps and settings.

The in-game Help panel provides a concise reference. This manual expands on it with detailed instructions, failure behaviour, function signatures and worked examples.

CHAPTER 17. TROUBLESHOOTING

17.1 Authentication and Account Issues

Table 17.1 lists the authentication failures that are implemented and their remedies. The exact player-facing strings are quoted from the account panel.

Table 17.1. Authentication troubleshooting.

Symptom	Cause	Resolution	Reference
Enter your username and password.	One or both fields empty.	Fill both fields and submit again.	Account panel message.
Usernames must be 3-24 characters: letters, numbers or underscore.	Username too short, too long or contains a space or symbol.	Use 3–24 letters, numbers or underscores.	Account panel message.
Passwords must be at least 6 characters.	Password shorter than 6.	Use at least 6 characters.	Account panel message.
Passwords can be at most 72 characters.	Password longer than 72.	Shorten the password.	Account panel message.
The passwords do not match.	Confirmation differs from password.	Re-enter the same password in both fields.	Account panel message.
That username is already taken.	Availability check found the username.	Choose another username.	Account panel message.
Incorrect username or password.	Generic credential rejection.	Re-enter the credentials; the message deliberately does not reveal whether the username exists.	Account panel message.
Too many attempts. Wait a moment and try again.	Too many attempts in a short time.	Wait and retry.	Account panel message.
The online service is temporarily unavailable.	The online service is temporarily unavailable.	Retry later.	Account panel message.
Could not reach SandEE online. Check your connection and try again.	Network failure or offline.	Check the connection and press Retry or reopen the account panel.	In-game message. shows the offline gate.

Symptom	Cause	Resolution	Reference
Account created, but sign-in was not completed. Try signing in again.	The online service requires confirmation and did not sign the account in.	Sign in again after confirmation.	Account panel message.
Leaderboard or cloud save opens a sign-in form.	No live session.	Sign in first.	Chapter 4
Password recovery is unavailable.	No recovery interface is implemented.	No in-game resolution exists currently.	Account panel.

17.2 World and Save Issues

Table 17.2 covers world selection, creation and saving.

Table 17.2. World and save troubleshooting.

Symptom	Cause	Resolution	Reference
Continue is disabled.	No readable world for the verified account.	Open Worlds and create or select a world.	Figure 5.1
Offline play cannot create new worlds. Reconnect to add a world.	Creation is online-only.	Reconnect and sign in, or play an existing world.	Figure 5.2
That name is already used by another world.	Duplicate name.	Choose a unique name.	In-game message.
Enter a name for your world.	Empty name.	Type a name.	World-manager message.
Create is disabled at ten worlds.	Per-account cap reached.	Delete a world before creating another.	World-manager behaviour.
World card missing after a cloud download.	Ownership sidecar or checksum mismatch.	Refresh the world list; the game fails closed rather than showing an unowned world.	Cloud-save behaviour.

Symptom	Cause	Resolution	Reference
Save file left as .tmp or .bak.	Crash during atomic write.	The next successful load uses the intact file; the temporary file is not treated as a world.	World-manager behaviour.
Autosave indicator does not appear.	No active slot or the loading screen is up.	Wait until the world is playable; autosave runs every 60 seconds.	Figure 6.1 shows the HUD context.

17.3 Inventory, Placement and Interaction Issues

Table 17.3 covers common gameplay failures.

Table 17.3. Gameplay troubleshooting.

Symptom	Cause	Resolution	Reference
New item remains on the ground.	Inventory and hotbar full.	Free a slot or move a stack to a chest.	Inventory overflow behaviour.
Reward is dropped beside the player.	Inventory full when a mission or dungeon reward was granted.	Pick it up after freeing space.	Reward overflow behaviour.
Red placement ghost.	Tile occupied, in water, too far or blocked by the player.	Move to a valid tile within six blocks.	Figure 7.1
Point-and-click order rejected.	No walkable approach to the target.	Move closer or clear the obstruction.	Pathfinding rule.
Too far to cast the rod reaches 8 blocks.	Fishing target beyond range.	Cast at water within eight blocks.	Fishing range rule.
Too far from water soil can only be tilled within 20 blocks of water.	Hoe used too far from water.	Move the plot closer to water.	Farming rule.

Symptom	Cause	Resolution	Reference
Crop does not harvest.	Wheat is still a sprout.	Wait for the 180-second growth time.	Crop growth rule.
Furnace does not smelt.	Missing input, fuel or output space.	Load both slots and empty the output.	Figure 11.2
Craft button reads MISSING MATERIALS.	Missing ingredients.	Gather the listed ingredients.	Figure 10.2
Chest opens but drag does not transfer.	Drop target is not a compatible slot.	Drag onto an empty or compatible slot.	Figure 7.2
Death does not delete inventory.	Intended behaviour.	Respawn at the active Bonfire or world spawn.	Figure 6.4
Creature shows a health bar after a hit.	Damage feedback.	Continue attacking or retreat.	Figure 9.5

17.4 Dungeon and Progression Issues

Table 17.4 covers dungeon entry, quiz and results.

Table 17.4. Dungeon troubleshooting.

Symptom	Cause	Resolution	Reference
Dungeon portal does nothing.	Player not nearby, build mode active, another UI open or pause open.	Leave build mode and close panels, then click the gate.	Figure 12.1
Quiz will not advance.	Output does not match expected text.	Compare the output console with the question; run again.	In-game message.
Hint reduces stars.	Each hint costs one star after the first dungeon.	Use hints only when needed.	In-game message.
Results show fewer than three stars.	Hints or wrong runs were used.	Replay the dungeon for a better score if desired.	Figure 12.3
Done opens a summary instead of the world.	The learning summary is shown after the results.	Press CONTINUE on the summary.	In-game message.
Dungeon completion does not remove the entrance.	The clear is recorded, but the entrance removal occurs on the exit/save path.	Return to the world through the summary and allow the save to complete.	In-game message.

CHAPTER 18. CONCLUSION

SandEE brings gathering, building, survival, exploration and programming together in one generated world. This manual has followed the full range of play: start a world, learn the controls, gather and craft, survive the overworld, and clear the Python and Lua dungeon tracks.

The order matters less than the habit. Read the tracker, keep your gear current, use the knowledge room to heal, and treat stars as a score rather than a gate: a zero-star clear still advances the missions. Once both tracks are complete, the side missions and the two programming environments give the sandbox its long tail.

The control chapters cover keyboard, mouse and touch play; the programming chapters list every custom Automatica and Algorithmica function. Players can read the manual in order or jump to the chapter they need.

CHAPTER 19. APPENDICES

Appendix A. Glossary

Table 19.1. Glossary of terms used in this manual.

Term	Meaning
Automatica	The programmable Lua robot and its editor.
Algorithmica	The Python-subset programming interface for crafting stations and furnaces.
Knowledge Altar	The dungeon object that records a Knowledge Entry.
Quiz Altar	The dungeon object that presents questions and clears the dungeon.
Blue star	A dungeon-performance star awarded for a clean run.
Yellow star	A mission reward star.
Skyglass Waypoint	A persistent overworld fast-travel tower.
Pixie Cave	A purple cave type, safe on first visit.
PNC	Point-and-click movement.
HUD	Heads-up display.

APPENDIX D. SURVEY

Questionnaire for IT Student Respondents

Name: _____ Age: _____

FUNCTIONAL SUITABILITY

Questions	1 Poorly Functional	2 Moderately Functional	3 Functional	4 Very Functional
Functional completeness - Degree to which the set of functions covers all the specified tasks and user objectives.				
Functional correctness - Degree to which a product or system provides the correct results with the needed degree of precision.				
Functional appropriateness - Degree to which the functions facilitate the accomplishment of specified tasks and objectives.				

PERFORMANCE EFFICIENCY

Questions	1 Poorly Efficient	2 Moderately Efficient	3 Efficient	4 Very Efficient
Time behaviour - Degree to which the response and processing times and throughput rates of a system, when performing its functions, meet requirements.				
Resource utilization - Degree to which the amounts and types of resources used by a system, when performing its functions, meet requirements.				
Capacity - Degree to which the maximum limits of a product or system parameter meet requirements.				

USABILITY

Questions	1 Poorly Usable	2 Moderately Usable	3 Usable	4 Very Usable
Appropriateness recognizability - Degree to which users can recognize whether a system is appropriate for their needs.				
Learnability - Degree to which a system can be used by specified users to achieve specified goals of learning to use the product or system with effectiveness, efficiency, freedom from risk and satisfaction in a specified context of use.				
Operability - Degree to which a system has attributes that make it easy to operate and control.				
User error protection - Degree to which a system protects users against making errors.				
User interface aesthetics - Degree to which a user interface enables pleasing and satisfying interaction for the user.				
Accessibility - Degree to which a system can be used by people with the widest range of characteristics and capabilities to achieve a specified goal in a specified context of use.				

PART 4: FEEDBACK AND SUGGESTIONS

Kindly identify its strong point/s to be enhanced and its weak points that need to be addressed for improvements.

_____

_____

_____

Do you have any thoughts on other features or functions that can be added to SandEE? Kindly share any suggestions with us so we can better improve it.

_____

_____

_____

Respondent's Signature: _____ Date of Evaluation: _____

Questionnaire for IT Experts

PART 1: RESPONDENT'S PROFILE

Name: _____

Age: _____

Affiliated Institution:

_____

Position/Designation:

_____

PART 2: SOFTWARE QUALITY EVALUATION

Directions: Please evaluate the software quality of SandEE based on the eight main characteristics of the ISO 25010 Software Product Quality Standards. Choose the appropriate box using the four-point scale with its corresponding verbal descriptions (4 as the highest and 1 as the lowest).

FUNCTIONAL SUITABILITY

Questions	1 Poorly Functional	2 Moderately Functional	3 Functional	4 Very Functional
Functional completeness - Degree to which the set of functions covers all the specified tasks and user objectives.				
Functional correctness - Degree to which a product or system provides the correct results with the needed degree of precision.				
Functional appropriateness - Degree to which the functions facilitate the accomplishment of specified tasks and objectives.				

PERFORMANCE EFFICIENCY

Questions	1 Poorly Efficient	2 Moderately Efficient	3 Efficient	4 Very Efficient
Time behaviour - Degree to which the response and processing times and throughput rates of a product or system, when performing its functions, meet requirements.				
Resource utilization - Degree to which the amounts and types of resources used by the system, when performing its functions, meet requirements.				

Capacity - Degree to which the maximum limits of the system parameter meet requirements.				

COMPATIBILITY

Questions	1 Poorly Compatible	2 Moderately Compatible	3 Compatible	4 Very Compatible
Co-existence - Degree to which a system can perform its required functions efficiently while sharing a common environment and resources with other products, without detrimental impact on any other product.				
Interoperability - Degree to which two or more systems, products or components can exchange information and use the information that has been exchanged.				

USABILITY

Questions	1 Poorly Usable	2 Moderately Usable	3 Usable	4 Very Usable
Appropriateness recognizability - Degree to which users can recognize whether a system is appropriate for their needs.				
Learnability - Degree to which a system can be used by specified users to achieve specified goals of learning to use the system with effectiveness, efficiency, freedom from risk and satisfaction in a specified context of use.				
Operability - Degree to which a system has attributes that make it easy to operate and control.				
User error protection - Degree to which a system protects users against making errors.				
User interface aesthetics - Degree to which a user interface enables pleasing and satisfying interaction for the user.				
Accessibility - Degree to which a system can be used by people with the widest range of characteristics and capabilities to achieve a specified goal in a specified context of use.				

RELIABILITY

Questions	1 Poorly Reliable	2 Moderately Reliable	3 Reliable	4 Very Reliable
Maturity - Degree to which a system or component meets needs for reliability under normal operation.				
Availability - Degree to which a system or component is operational and accessible when required for use.				
Fault tolerance - Degree to which a system or component operates as intended despite the presence of hardware or software faults.				
Recoverability - Degree to which, in the event of an interruption or a failure, a system can recover the data directly affected and re-establish the desired state of the system.				

SECURITY

Questions	1 Poorly Secure	2 Moderately Secure	3 Secure	4 Very Secure
Confidentiality - Degree to which a system ensures that data are accessible only to those authorized to have access.				
Integrity - Degree to which a system component prevents unauthorized access to, or modification of, computer programs or data.				
Non-repudiation - Degree to which actions or events can be proven to have taken place so that the events or actions cannot be repudiated later.				
Accountability - Degree to which the actions of an entity can be traced uniquely to the entity.				
Authenticity - Degree to which the identity of a subject or resource can be proved to be the one claimed.				

MAINTAINABILITY

Questions	1 Poorly Maintainable	2 Moderately Maintainable	3 Maintainable	4 Very Maintainable
Modularity - Degree to which a system or computer program is composed of discrete components such that a change to one component has minimal impact on other components.				
Reusability - Degree to which an asset can be used in more than one system, or in building other assets.				
Analysability - Degree of effectiveness and efficiency with which it is possible to assess the impact on a product or system of an intended change to one or more of its parts, or to diagnose a product for deficiencies or causes of failures, or to identify parts to be modified.				
Modifiability - Degree to which a product or system can be effectively and efficiently modified without introducing defects or degrading existing product quality.				
Testability - Degree of effectiveness and efficiency with which test criteria can be established for a system, product or component and tests can be performed to determine whether those criteria have been met.				

PORTABILITY

Questions	1 Poorly Portable	2 Moderately Portable	3 Portable	4 Very Portable
Adaptability - Degree to which a product or system can effectively and efficiently be adapted for different or evolving hardware, software or other operational or usage environments.				
Installability - Degree of effectiveness and efficiency with which a product or system can be successfully installed and/or uninstalled in a specified environment.				
Replaceability - Degree to which a product can replace another specified software product for the same purpose in the same environment.				

PART 4: FEEDBACK AND SUGGESTIONS

Kindly identify its strong point/s to be enhanced and its weak points that need to be addressed for improvements.

_____
_____
_____

Do you have any thoughts on other features or functions that can be added to SandEE? Kindly share any suggestions with us so we can better improve it.

_____
_____
_____

Respondent's Signature: _____ Date of Evaluation: _____

- If progress is lost, check the save slot and the cloud-save menu before creating a new world.

APPENDIX E. SOURCE CODE

E.1 Interpreter Implementation

SandEE embeds its own language runtime as a Rust GDExtension. Lua programs are executed through the mlua library, while Python programs are parsed with ruff_python_parser and transpiled to GDScript before execution. The listings below retain the original source line numbers and show the core execution and translation paths.

E.1.1 Lua Interpreter

Listing E.1. Lua execution entry point and sandbox restrictions (extension/src/runner/lua.rs)

```
275     fn run_code(&mut self) {
276         if let Err(e) = self._run_code(self.code.to_string().as_ref()) {
277             self.signals().code_failed().emit(e.to_string());
278         } else {
279             self.signals().code_executed().emit();
280         }
281     }
282
283     // Every item display name lets a GDScript editor offer a name popup.
284     #[func]
285     fn run_list_items(&mut self) -> PackedStringArray {
286         let mut names: Vec<String> = crate::utils::item_names();
287         names.sort();
288         let mut out = PackedStringArray::new();
289         for n in names {
290             out.push(&GString::from(n.as_str()));
291         }
292         out
293     }
294
295     fn _run_code(&mut self, code: &str) -> Result<()> {
296         // set variables that the user must be able to retrieve
```

```
297         let globals = self.lua.globals();
298         if self.held_item.is_empty() {
299             globals.set("held_item", Value::Nil)?;
300         } else {
301             let held_item = self.lua.create_table()?;
302             held_item.set("name", self.held_item.to_string())?;
303             held_item.set("quantity", self.held_item_quantity)?;
304             globals.set("held_item", held_item)?;
305         }
306         // Crafting Station state, so a program can read it (station_recipe,
307         // station_output_item, station_output_count)  see LuaState's #[var]s.
308         globals.set("station_recipe", self.station_recipe.to_string())?;
309         globals.set("station_output_item", self.station_output_item.to_string())?;
310         globals.set("station_output_count", self.station_output_count)?;
311         self.lua.load(code).exec()?;
312         Ok(())
313     }
    ...
563     fn restrict_globals(&mut self) -> Result<()> {
564         const ALLOWED: &[&str] = &[
565             // language built-ins
566             "print", "type", "tostring", "tonumber", "pairs", "ipairs", "next",
567             "select", "error", "assert", "pcall", "xpcall", "unpack",
568             "rawget", "rawset", "rawequal", "rawlen", "setmetatable",
569             "getmetatable", "math", "string", "table", "coroutine", "utf8",
570             // game globals + custom API
571             "vector", "held_item", "station_recipe", "station_output_item",
572             "station_output_count", "GODOT_PROP_signals", "move", "attack",
573             "interact", "drop_item", "place_item", "place_at", "select_block",
574             "withdraw", "craft", "set_recipe", "take_output", "put_input",
575             "input_slot", "mine", "rename", "combat_mode", "list_items",
```

```
576         ];
577         let globals = self.lua.globals();
578         let mut remove: Vec<String> = Vec::new();
579         for pair in globals.clone().pairs::<String, Value>() {
580             let (key, _) = pair?;
581             if !ALLOWED.contains(&key.as_str()) {
582                 remove.push(key);
583             }
584         }
585         for key in remove {
586             globals.set(key, Value::Nil)?;
587         }
588         Ok(())
589     }
```

E.1.2 Python Interpreter

Listing E.2. Python parsing and Python-to-GDScript transpilation (extension/src/transpiler/transpile.rs)

```
125     fn transpile_statement(&mut self, statement: &mut Stmt) -> Result<String> {
126         let transpiled = match statement {
127             Stmt::FunctionDef(func) => {
128                 let mut transpiled = format!("func {}(", func.name.as_str());
129                 let (args, kwargs, final_args, args_spec) = self.build_function_args(func);
130 
131                 transpiled.push_str(&final_args.join(", "));
132                 transpiled.push_str("):\n");
133 
134                 self.symbols
135                     .add_symbol(Symbol::Function(func.name.to_string(), args_spec));
136 
137                 self.indent();
138                 if let Some(kwargs_name) = func.parameters.kwarg.as_ref() {
139                     self.symbols.add_symbol(Symbol::Variable(
```

```
                kwargs_name.name().to_string(),
                PythonType::Dict,
            ));
        }

        args.iter().for_each(|arg| {
            self.symbols
                .add_symbol(Symbol::Variable(arg.to_string(), PythonType::Any))
        });

        for (key, value) in kwargs {
            self.symbols.add_symbol(Symbol::Variable(
                key.to_string(),
                PythonType::from_expr(value, &self.symbols)?,
            ));
            transpiled.push_str(&format!(
                "{}{} = {} if {} else {}\n",
                self.indent_string(),
                key,
                key,
                key,
                self.transpile_expression(value)?
            ));
        }

        if let Some(vararg) = func.parameters.vararg.as_ref() {
            self.symbols.add_symbol(Symbol::Variable(
                vararg.name().to_string(),
                PythonType::List,
            ));
        }
```

```
171
172                 transpiled.push_str(&self.transpile_statements(&mut func.body)?);
173                 self.dedent();
174
175                 Ok(transpiled)
176             }
    ...
758     fn transpile(&mut self, code: &str) -> Result<String> {
759         self.symbols.reset();
760                                     let mut ast = ruff_python_parser::parse(code,
ParseOptions::from(Mode::Module))?
761             .try_into_module()
762             .ok_or(anyhow!("Failed to convert into Python module."))?
763             .into_suite();
764         let slice = ast.as_mut_slice();
765         let transpiled = self.transpile_statements(slice)?;
766         if self.symbols.exists("main") {
767             Ok(transpiled)
768         } else {
769             Err(anyhow!("No main function found"))
770         }
771     }
772 }
773
774 #[derive(GodotClass)]
775 #[class(base=Node, init)]
776 struct PythonTranspiler {
777     transpiler: Transpiler,
778     base: Base<Node>,
779 }
780
```

```
#[godot_api]
impl PythonTranspiler {
    #[func]
    fn transpile(&mut self, code: String) -> Array<Variant> {
        let (ok, transpiled) = match self.transpiler.transpile(&code) {
            Ok(transpiled) => (true, transpiled),
            Err(error) => (false, format!("{:?}", error)),
        };
        let mut ret: Array<Variant> = Array::new();
        ret.push(ok);
        ret.push(transpiled);
        ret
    }
```

E.2 Account Implementation

The account system combines a reusable account panel, a Supabase authentication client, encrypted local session storage, and an account-scoped world access layer. The excerpts show the login and registration flow, server-side authentication calls, session restoration, and the checks that prevent one account from accessing another account's worlds.

E.2.1 Account Interface

Listing E.3. Login, registration, and sign-out handlers (scripts/online_account_panel.gd)

```
436  func _submit_login() -> void:
437      if _busy:
438          return
439      var user_value: String = (_fields["username"] as LineEdit).text.strip_edges()
440      var password: String = (_fields["password"] as LineEdit).text
441      if user_value.is_empty() or password.is_empty():
442          _show_error("Enter your username and password.")
443          return
444      _set_busy(true, "Signing in…")
445      var result: Dictionary = await Supabase.sign_in(user_value, password)
446      if not result.ok:
447          _set_busy(false)
448          _show_error(_auth_error_text(result))
449          return
450      await Supabase.refresh_profile()
451      _clear_fields()
452      _set_busy(false)
453      authenticated.emit()
454
455
456  func _submit_register() -> void:
457      if _busy:
```

```
458         return
459     var user_value: String = (_fields["username"] as LineEdit).text.strip_edges()
460     var password: String = (_fields["password"] as LineEdit).text
461     var confirm: String = (_fields["confirm"] as LineEdit).text
462     if not _valid_username(user_value):
463                 _show_error("Usernames must be 3-24 characters: letters, numbers or underscore.")
464         return
465     if password.length() < PASSWORD_MIN:
466       _show_error("Passwords must be at least %d characters." % PASSWORD_MIN)
467         return
468     if password.length() > PASSWORD_MAX:
469       _show_error("Passwords can be at most %d characters." % PASSWORD_MAX)
470         return
471     if password != confirm:
472         _show_error("The passwords do not match.")
473         return
474
475     _set_busy(true, "Checking username…")
476     var available := await Supabase.username_available(user_value)
477     if Supabase.last_username_check_error != "":
478         _set_busy(false)
479         _show_error(_network_error_text())
480         return
481     if not available:
482         _set_busy(false)
483         _show_error("That username is already taken.")
484         return
485
486     _set_busy(true, "Creating account…")
487     var result: Dictionary = await Supabase.register(user_value, password)
```

```
    if not result.ok:
        _set_busy(false)
        _show_error(_register_error_text(result))
        return
    _clear_fields()
    if not Supabase.is_signed_in():
        # The project can require email confirmation; no session was issued.
        _set_busy(false)
        open_login("Account created, but sign-in was not completed. Try signing in again.")
        return
    await Supabase.refresh_profile()
    _set_busy(false)
    authenticated.emit()


func _submit_logout() -> void:
    if _busy:
        return
    _set_busy(true, "Signing out…")
    # Invalidate the active account context BEFORE the token is cleared, so no
    # autosave/cloud callback can still treat the old account as current.
    Global.begin_account_transition("sign_out")
    await Supabase.sign_out()
    Global.clear_active_account("sign_out")
    _set_busy(false)
    logged_out.emit()
```

E.2.2 Authentication Client

Listing E.4. Registration, sign-in, refresh, and session validation (scripts/supabase_client.gd)

```
188 func sign_up(email: String, password: String, username: String) -> Dictionary:
189     var result := await _request("POST", "/auth/v1/signup", {
190         "email": email,
191         "password": password,
192         "data": {"username": username},
193     }, false)
194     if result.ok:
195         _apply_session(result.data)
196     return result
197
198
199 ## Registers with only a username and password. Supabase Auth needs an
200 ## identifier, so it is derived privately from the username
201 ## (`<lowercase username>@users.sandee.invalid`) and is never collected, shown
202 ## `.invalid` is reserved by RFC 2606.
203 func register(username_value: String, password: String) -> Dictionary:
204     var uname := username_value.strip_edges()
205                         var account_email := uname.to_lower() + "@" +
_CONFIG.ACCOUNT_EMAIL_DOMAIN
206     return await sign_up(account_email, password, uname)
207
208
209 ## Signs in with a username through the `login-with-username` Edge Function.
210 func sign_in(username_value: String, password: String) -> Dictionary:
211     var result := await _request("POST", "/functions/v1/login-with-username", {
212         "username": username_value,
213         "password": password,
214     }, false)
215     if result.ok:
```

```
        _apply_session(result.data)
    return result


## password fallback if the Edge Function is unavailable.
func sign_in_with_email(email: String, password: String) -> Dictionary:
    var result := await _request("POST", "/auth/v1/token?grant_type=password", {
        "email": email,
        "password": password,
    }, false)
    if result.ok:
        _apply_session(result.data)
    return result


func refresh_session(timeout: float = -1.0) -> bool:
    if refresh_token == "":
        return false
    var result := await _request("POST", "/auth/v1/token?grant_type=refresh_token",
{
        "refresh_token": refresh_token,
    }, false, {}, timeout)
    if result.ok:
        _apply_session(result.data)
        return true
    # A network outage must NOT destroy a valid refresh token: only a definitive
    # auth rejection (4xx) clears the stored session.
    if result.status >= 400 and result.status < 500:
        _clear_session()
    return false

```

```
246
247 ## Restores and validates the stored session against the live backend.
248 ## Returns true only when the profile row was actually retrieved. The short
249 ## default timeout keeps the startup screen from waiting on a dead session.
250 func validate_session(timeout: float = 6.0) -> bool:
251     if not await ensure_session(timeout):
252         return false
253     var result := await get_profile(timeout)
254     if result.ok and result.data is Array and (result.data as Array).size() == 1:
255         profile = (result.data as Array)[0]
256         return true
257     if result.status == 401 or result.status == 403:
258         _clear_session()
259     return false
260
261
262 ## Refreshes the cached profile row (username + star totals).
263 func refresh_profile() -> Dictionary:
264     var result := await get_profile()
265     if result.ok and result.data is Array and (result.data as Array).size() == 1:
266         profile = (result.data as Array)[0]
267     return result
268
269
270 ## Returns true when a usable access token is available (refreshing if needed).
271 func ensure_session(timeout: float = -1.0) -> bool:
272     if access_token == "":
273         return false
274     if expires_at <= 0.0:
275         return true
276     if Time.get_unix_time_from_system() + _CONFIG.REFRESH_MARGIN_SEC <
```

```
expires_at:
277         return true
278     return await refresh_session(timeout)
```

E.2.3 Session Persistence

Listing E.5. Encrypted local session storage (scripts/supabase_client.gd)

```
78  func _session_pass() -> String:
79      var device_id := OS.get_unique_id().strip_edges()
80      if device_id.is_empty():
81          device_id = str(ProjectSettings.globalize_path("user://"))
82      return "SandEE|cloud-session|v1|" + device_id + "|" + OS.get_processor_name()
83
84
85  func _load_session() -> void:
86      _remove_legacy_session_file()
87      if not FileAccess.file_exists(_CONFIG.SESSION_PATH):
88          return
89      var file := FileAccess.open_encrypted_with_pass(
90          _CONFIG.SESSION_PATH, FileAccess.READ, _session_pass()
91      )
92      if file == null:
93          _wipe_session_file()
94          return
95      var text := file.get_as_text()
96      file.close()
97      var parsed: Variant = JSON.parse_string(text)
98      if typeof(parsed) != TYPE_DICTIONARY:
99          _wipe_session_file()
100         return
101     access_token = str(parsed.get("access_token", ""))
102     refresh_token = str(parsed.get("refresh_token", ""))
103     expires_at = float(parsed.get("expires_at", 0.0))
```

```
104     var stored_user: Variant = parsed.get("user", {})
105     if typeof(stored_user) == TYPE_DICTIONARY:
106         user = stored_user
107
108
109 func _save_session() -> void:
110     if access_token == "":
111         _clear_session()
112         return
113     var tmp_path := _CONFIG.SESSION_PATH + ".tmp"
114      var file := FileAccess.open_encrypted_with_pass(tmp_path, FileAccess.WRITE,
_session_pass())
115     if file == null:
116         return   # never fall back to plaintext
117     file.store_string(JSON.stringify({
118         "access_token": access_token,
119         "refresh_token": refresh_token,
120         "expires_at": expires_at,
121         "user": user,
122     }))
123     file.close()
124     var abs_path := ProjectSettings.globalize_path(_CONFIG.SESSION_PATH)
125     var abs_tmp := ProjectSettings.globalize_path(tmp_path)
126     DirAccess.remove_absolute(abs_path)
127     if DirAccess.rename_absolute(abs_tmp, abs_path) != OK:
128         DirAccess.remove_absolute(abs_tmp)
    ...
145 func _clear_session() -> void:
146     var was_signed_in := is_signed_in()
147     access_token = ""
148     refresh_token = ""
```

```
    expires_at = 0.0
    user = {}
    profile = {}
    _wipe_session_file()
    if was_signed_in:
        auth_changed.emit(false)


func _apply_session(payload: Dictionary) -> void:
    var session: Dictionary = payload
    if typeof(payload.get("session")) == TYPE_DICTIONARY:
        session = payload["session"]
    if payload.has("access_token"):
        access_token = str(payload["access_token"])
    elif session.has("access_token"):
        access_token = str(session["access_token"])
    if payload.has("refresh_token"):
        refresh_token = str(payload["refresh_token"])
    elif session.has("refresh_token"):
        refresh_token = str(session["refresh_token"])
    var expires_in := float(payload.get("expires_in", session.get("expires_in", 3600)))
    expires_at = Time.get_unix_time_from_system() + expires_in
    var payload_user: Variant = payload.get("user", null)
    if typeof(payload_user) == TYPE_DICTIONARY:
        # A different account must never inherit the previous profile cache.
        if str(user.get("id", "")) != str((payload_user as Dictionary).get("id", "")):
            profile = {}
        user = payload_user
    elif typeof(session.get("user")) == TYPE_DICTIONARY:
        user = session["user"]
    _save_session()
```

```
180     auth_changed.emit(true)
```

E.2.4 Startup Flow and Account Context

Listing E.6. Startup session restoration and account entry (scripts/startup_mode.gd)

```
406 func _on_online() -> void:
407     if _busy or _leaving or (_online_btn != null and _online_btn.disabled):
408         return
409     _busy = true
410     _set_mode_buttons_enabled(false)
411     _set_progress(true, 5.0)
412     _set_status("Checking online services…", C_MUTED)
413
414     var connectivity := await _probe_connectivity()
415     if not is_inside_tree():
416         return
417     if not connectivity.get("backend", false):
418         _connectivity = connectivity
419         var offline_id := Global.offline_account_id()
420         var cached_id := Supabase.user_id()
421         var cached_matches: bool = cached_id.is_empty() or cached_id == offline_id
422         if not offline_id.is_empty() and cached_matches \
423                 and Global.restore_offline_account(offline_id):
424             _busy = false
425             _set_progress(true, 100.0, "Playing your verified offline copy...")
426             await _go_to_menu()
427             return
428         _busy = false
429         _set_progress(false)
430         _set_mode_buttons_enabled(true)
431         _apply_connectivity()
432         return
433
```

```
434     if Global.has_incomplete_account_transition():
435         # An interrupted sign-out/switch must never restore the previous
436         # session. Clear it and require a fresh sign-in.
437         await Supabase.sign_out()
438         if not is_inside_tree():
439             return
440         _busy = false
441         _set_progress(false)
442         _set_mode_buttons_enabled(true)
443         _show_login("Finish signing in to continue.")
444         return
445
446     if Supabase.has_stored_session():
447         _set_status("Restoring session…", C_MUTED)
448         var restored := await Supabase.validate_session()
449         if not is_inside_tree():
450             return
451         if restored:
452             _busy = false
453             await _enter_online()
454             return
455         # The stored token is expired/invalid: clear it before showing the form.
456         await Supabase.sign_out()
457         if not is_inside_tree():
458             return
459
460     _busy = false
461     _show_login("Sign in to play SandEE. An online account is required.")
    ...
529     if not Global.begin_account_session(Supabase.user_id()):
530         _busy = false
```

```
531         _set_progress(false)
532         _set_mode_buttons_enabled(true)
533         _show_login("Your account identity was invalid. Sign in again.")
534         return
```

Listing E.7. Account-scoped session and world access control (scripts/global.gd)

```
1624  func can_access_worlds() -> bool:
1625      if account_transitioning or not is_valid_account_id(current_account_id):
1626          return false
1627      if online_mode:
1628          return account_online_verified and account_cloud_verified
1629      return account_offline_verified
1630
1631  func _identity_is_verified() -> bool:
1632      return not account_transitioning and is_valid_account_id(current_account_id) \
1633          and (account_online_verified or account_offline_verified)
     ...
1670  func begin_account_session(account_id: String, remember: bool = true) -> bool:
1671      var id := account_id.strip_edges().to_lower()
1672      if not is_valid_account_id(id):
1673          clear_active_account("invalid_account")
1674          return false
1675      if is_valid_account_id(current_account_id) and current_account_id != id:
1676          begin_account_transition("account_switch")
1677      _clear_active_world_state()
1678      if not _ensure_account_dir(id):
1679          clear_active_account("storage_unavailable")
1680          return false
1681      current_account_id = id
1682      account_online_verified = true
1683      account_offline_verified = false
1684      account_cloud_verified = false
```

```
1685      # The persisted transition marker still exists until the cloud sync succeeds,
1686      # but the in-memory flag must not block the sync itself. World access remains
1687      # closed because account_cloud_verified is false.
1688      account_transitioning = false
1689      if remember:
1690          last_verified_account_id = id
1691          _write_account_binding(id)
1692      account_context_changed.emit(current_account_id)
1693      return true
      ...
1717  func begin_account_transition(reason: String = "switch") -> void:
1718      if is_valid_account_id(current_account_id):
1719          _write_transition_marker(reason)
1720      account_transitioning = true
1721      _remove_account_binding()
1722      clear_active_account(reason)
1723
1724  func finish_account_transition() -> void:
1725      account_transitioning = false
1726      _remove_transition_marker()
1727
1728  ## Clears the active account and all world/UI state. Does not delete account
1729  ## files and does not by itself clear the offline binding (begin_account_transition
1730  ## does that for explicit sign-out/switch).
1731  func clear_active_account(reason: String = "cleared") -> void:
1732      current_account_id = ""
1733      account_online_verified = false
1734      account_offline_verified = false
1735      account_cloud_verified = false
1736      _clear_active_world_state()
1737      account_context_changed.emit("")
```

```
1738    if reason != "":
1739        CrashLogger.event("ACCOUNT", "active account cleared (%s)" % reason)
```

APPENDIX F. CURRICULUM VITAE OF RESEARCHERS

Curriculum Vitae of

Accad, Glenn Marco B.

Block 32, Lot 17 Phase 6C Towerville, Barangay Gaya - Gaya, Towerville, San Jose Del Monte Bulacan
Accad.355761@sjdelmonte.sti.edu.ph
+639981847987

EDUCATIONAL BACKGROUND

Level	Inclusive Dates	Name of school/ Institution
Tertiary	August 2023	STI College San Jose Del Monte
Vocational/Technical		
High School	May 2017-2023	Graceville National High school
Elementary	June 2011-2017	Marangal Elementary School

PROFESSIONAL OR VOLUNTEER EXPERIENCE

Inclusive Dates	Nature of Experience Job Title	Name and Address of Company or Organization
February 2023 – May 2023	Student Assistant	San Jose Academy of Bulacan

AFFILIATIONS

Inclusive Dates	Name of Organization	Position
August 2023-2024	Intellitech	Member
August 2024-2025	Intellitech	Member
August 2025-2026	Intellitech	Member

SKILLS

SKILLS	Level of Competency	Date Acquired
MS Office	Intermediate	June 2011
Java Programming	Intermediate	October 2023
C# Programming	Intermediate	May 2024

TRAININGS, SEMINARS OR WORKSHOP ATTENDED

Inclusive Dates	Title of Training, Seminar or Workshop
August 2023	**National Service Training Program**
February 2024	**GitGud**
	CodeFest

Curriculum Vitae of

Gallano, Adrian V.

1067 Malaya Street, Sapang Alat Road, Barangay 181, Pangarap Village, Caloocan City, NCR
gallano.360632@sjdelmonte.sti.edu.ph
+63927208694

EDUCATIONAL BACKGROUND

Level	Inclusive Dates	Name of school/ Institution
Tertiary	August 2023	STI College San Jose Del Monte
Vocational/Technical		
High School	July 2017-2023	Immaculada Conception Colleges
Elementary	August 2011-2017	Our Lady of Mercy School

PROFESSIONAL OR VOLUNTEER EXPERIENCE

Inclusive Dates	Nature of Experience Job Title	Name and Address of Company or Organization

AFFILIATIONS

Inclusive Dates	Name of Organization	Position
August 2023-2024	Intellitech	Member
August 2024-2025	Intellitech	Member
August 2025-2026	Intellitech	Member

SKILLS

SKILLS	Level of Competency	Date Acquired
MS Office	Intermediate	June 2018
Java Programming	Intermediate	October 2023
C# Programming	Intermediate	September 2024

TRAININGS, SEMINARS OR WORKSHOP ATTENDED

Inclusive Dates	Title of Training, Seminar or Workshop
August 2023	National Service Training Program
February 2024	GitGud

Curriculum Vitae of

MAMAUAG, JEFFERSON G.

Block 66 Lot 24 Nuvista Subdivision Barangay Santo Cristo City of San Jose Del Monte Bulacan
mamauag.248577@sjdelmonte.sti.edu.ph
0931096819

EDUCATIONAL BACKGROUND

Level	Inclusive Dates	Name of school/ Institution
Tertiary	August 2023	STI College San Jose Del Monte
Vocational/Technical		
High School	May 2016-2023	Yverdon De Pestalozzi School
Elementary	June 2010-2016	Placido Del Mundo Elementary School

PROFESSIONAL OR VOLUNTEER EXPERIENCE

Inclusive Dates	Nature of Experience Job Title	Name and Address of Company or Organization

AFFILIATIONS

Inclusive Dates	Name of Organization	Position
July 2021-2023	Code Camp Society	Member
August 2023-2024	Intellitech	Member
August 2024-2025	Intellitech	CS 2nd Year representative
August 2025-2026	Intellitech	Member

SKILLS

SKILLS	Level of Competency	Date Acquired
MS Word	Intermediate	September 2016
Java Programming	Intermediate	December 2023
Python Programming	Intermediate	April 2025

TRAININGS, SEMINARS OR WORKSHOP ATTENDED

Inclusive Dates	Title of Training, Seminar or Workshop
August 2023	National Service Training Program
February 2024	GitGud

Curriculum Vitae of

Tayag, Jose Mari Christian V.

Block 34, Lot 5, Saint Therese Phase, Deca Homes Subdivision, Barangay Loma de Gato, Marilao, Bulacan
tayag.334839@sjdelmonte.sti.edu.ph
+639166248648

EDUCATIONAL BACKGROUND

Level	Inclusive Dates	Name of school/ Institution
Tertiary	**August 2023**	**STI College San Jose Del Monte**
Vocational/Technical		
High School	**May 2014-2020**	**Caloocan National Science and Technology Highschool**
Elementary	**June 2008-2014**	**ST. Anne Power Academy-Marilao, inc.**

AFFILIATIONS

Inclusive Dates	Name of Organization	Position
August 2023-2024	Intellitech	Member
August 2024-2025	Intellitech	Member
August 2025-2026	Intellitech	Member

SKILLS

SKILLS	Level of Competency	Date Acquired
Ux/Ui Design	Intermediate	September 2020
Python Programming	Intermediate	March 2018
Rust Programming	Intermediate	December 2024

TRAININGS, SEMINARS OR WORKSHOP ATTENDED

Inclusive Dates	Title of Training, Seminar or Workshop
August 2023	**National Service Training Program**
February 2024	**GitGud**
	CodeFest